Sparse Prefill Prediction: Research Plan and Task List

Status: active, single source of truth. **Last updated:** 19 October 2026. **Supersedes:** every earlier plan, including all versions of the "Estimator Frontier" plan. Their reasoning and referee responses are kept, unedited, in `docs/archive/` for traceability; nothing in them overrides this file.

---

0. How to use this document

Who reads it. The researcher, Claude chat (direction and decisions) and Claude Code (implementation). All three treat it as authoritative.

Rules.

1. If anything (code, an older doc, a chat message, memory) conflicts with this plan, this plan wins until it is changed. Flag the conflict; do not silently follow the other source.
2. The plan changes only by editing `docs/RESEARCH_PLAN.md` and committing. The PDF is regenerated from it in the same commit (`python tools/build_plan_pdf.py`). Never edit the PDF by hand.
3. Task status (Section 8) is updated in the same commit as the work that changes it.
4. A new decision gets a body change **and** one dated line in Appendix A. No version numbers.
5. Direction changes (gaps, hypotheses, pairings, arms, budget) are decided in Claude chat with the researcher, then written here. Claude Code implements what is written here and asks before changing scope.

Status words. *Done*: built, tested, used. *Built*: code and tests exist, not yet used for the study. *Partly built*: some pieces exist (named). *In progress*: started, nothing usable yet; the finished and remaining pieces are named. *Not started*. *Blocked* (reason given). *Decision*: needs the researcher.

---

1. The question

When does training-free block-sparse prefill actually pay end to end on a given host CPU and GPU, and can that be predicted before timing the workload there?

- **RQ1.** Can component measurements predict end-to-end prefill time, and the sparse-versus-dense crossover, on a pairing on which the workload has not been timed? (H1, H2, H3) No timing of the workload on the target is an input to its prediction; the operation order and the estimator's selection are recorded per code path, on the target when its code path is new, with every duration discarded (Section 5.1).
- **RQ2.** Does keeping the execution order and position of each estimator's host work and synchronisations improve that prediction over aggregate models? (H1 against the KernelSight-style step form and the fitted sum; the XAttention pair, where XAttention is Xu et al., "XAttention: Block Sparse Attention with Antidiagonal Scoring", ICML 2025; arXiv:2503.16428; code `mit-han-lab/x-attention`, used at commit `e379887`, installed and called, with the lines listed in `docs/vendor_register.md` reproduced)
- **RQ3.** At matched per-layer recall, is accuracy the same whichever estimator chose the blocks? (H4)
- **RQ4.** Do the model's predictions, combined with the admissible density, decide whether to use sparse prefill on an unmeasured pairing better than the decision suggested by kernel speedups? (H6)

Papers report sparse attention kernel speedups on one or two machines. Our own measurements show the kernel speedup often does not survive end to end. On an A100 with our reference mask builder, prefill became slower (0.63x to 0.75x) although the sparse kernel beat PyTorch's SDPA flash kernel. About 94% of the builder's loop time (a laptop profile at 32768 tokens, not banked; to be re-measured on the target host under T2.2) was Python overhead, and a vectorised builder turned the A100 result into a speedup; against the fastest correct dense kernel the speedup is smaller still, and at 8192 tokens it becomes a loss (Section 2). So host-side cost is not fixed: it depends on how an estimator is implemented. What remains when masks are built on the GPU is host-visible work and GPU-to-CPU synchronisation. Released research code still reads GPU values from Python: XAttention runs two `assert` checks on GPU tensors per query chunk in every layer, and the FlexPrefill implementation bundled with it sizes index tensors with `.max().item()`. Compilation does not remove all of these by default: under `torch.compile` with default settings, `.item()`, `nonzero()` and an `assert` that turns a tensor into a Python bool (such as `assert not t.any()`) each break the graph, while a plain `assert` on a tensor (rewritten into `torch._assert`) and a fixed-size `topk` do not (CPU, PyTorch 2.8.0 and 2.14.1; `results/compile_graph_breaks_cpu/`, `scripts/check_compile_graph_breaks.py`). XAttention's prefill path uses the plain form (upstream commit `e379887`, `xattn/src/utils.py`), one of the two inside a `try`/`except`; the plain form did not split the graph on a CPU, and the `and` form, which splits, is on the decoding path only. Whether the compiled code still makes the GPU wait for them is measured under T2.21. Compiled arms measure what remains on the GPU (Section 5.5). Whether sparse prefill pays therefore depends on the host, the GPU, how their work overlaps, which synchronisations an implementation makes, and whether the sparse masks keep accuracy.

Contributions, in order of weight.

- **G2 (lead). The cost of data-dependent sparsity.** Sparse attention keeps different blocks for every input, so an implementation must read sizes or checks back to the host (a synchronisation), pad to a worst case (extra GPU work), or work in fixed-size buffers (a fixed budget). G2 is a per-operation critical-path model of the order and position, *within* a prefill step, of each estimator's data-dependent host work and GPU-to-CPU synchronisations (mask building wherever it runs, data-dependent sizing, checks that read GPU values), built only from measured components. It prices synchronising and fixed-size implementations on a given pairing; where an implementation pads, the padded work is part of its device cost. Aggregate models, including KernelSight-LM's step formula, do not represent where in a step these fall (Section 3). It predicts what each synchronisation costs on a given pairing, and so what removing it would gain, before anyone builds a fused kernel. Arms without synchronisations, including compiled arms with no wait left, are part of the test: on them the model should add nothing over simpler forms. **Not claimed as new:** the host-cost measurement method, which adapts TaxBreak's null-kernel protocol (TaxBreak uses null kernels for the launch path; we substitute them across the whole operation sequence and measure host-side compute directly at target sizes), and the queue-aware recursion, which is a standard form of host-device overlap. The adaptation is not claimed as a contribution. What is new is placing data-dependent synchronisations within the step and predicting their cost on pairings where the workload has not been timed.
- **G3 (lead).** A test of whether attention recall is an estimator-independent proxy for accuracy: at matched per-layer recall, does accuracy depend on which estimator chose the blocks? This is tested, not assumed. Prior evidence points both ways: within its own indexer, VSPrefill finds accuracy indistinguishable from full attention above 90% recall and calls recall "a faithful surrogate objective"; in decoding, Less Is More argues that which tokens are kept (a stable recency window) matters. Either outcome of H4 is reported.
- **Application: a profitability dispatcher (RQ4, H6).** For each pairing, context length and batch, the dispatcher uses the predicted time and the admissible density to choose dense or a sparse arm at a density, before the workload is timed on that pairing (Section 5.9). It is scored by regret against the best measured choice and compared with the decision suggested by kernel speedups. It answers the question a practitioner faces on a given host and GPU: should sparse prefill be switched on at all?
- **G1 (validation question).** Whether this explicit model predicts end-to-end time on real, physically distinct host-GPU pairings better than a learned predictor when only a few hardware systems are available for training. Cross-device latency prediction itself is not new (CTFusion), so G1 is not claimed as a new problem. The fitted sum (Section 5.4) is the primary comparison for G1; the learned predictor is secondary.

Name. The method is named BUNGA (Block-sparse Utility Navigator for GPU Attention), made of the G2 critical-path model (components defined in Section 5.1, the recursion in Section 5.4) and the profitability dispatcher (Section 5.9), while `attnbench` remains the name of the measurement harness.

Scope. One GPU, one CUDA stream per forward pass, batch 1 and 4, PyTorch eager and compiled. Claims do not extend to multi-GPU, continuous batching or multi-stream serving engines; the abstract states this.

How the two halves connect. "Profitable" (Section 5.1) needs both a speedup and non-inferior accuracy. Predicting it on an unmeasured pairing therefore needs two things: end-to-end time at a given density (G2, RQ1 and RQ2) and the lowest density at which each estimator keeps accuracy. If H4 holds, that density can be read from recall, which is cheap to measure and does not depend on the hardware; combined with the time model, profitability on an unmeasured pairing is then predicted with no accuracy runs at all. If H4 fails, each estimator needs its own accuracy runs, and the study reports where recall falls short. Either way the result is one pipeline: predicting when sparse prefill pays (T7.6). The dispatcher (Section 5.9) turns that prediction into a decision and scores it (T7.7).

If sparse prefill never pays. In the pilots no deployable estimator was non-inferior at any tested density, so the profitable set may be empty on every pairing. That is stated in advance as a possible result, not a failure. The claims then are: predicted against measured end-to-end time and crossover per pairing and arm (RQ1, RQ2); which operations and synchronisations keep each arm from paying, and by how much on each pairing; whether the dispatcher avoids the losses the kernel-speedup rule would make (RQ4); and the recall result (RQ3). The paper's title would then name prediction, not profitability.

Target. An IEEE Transactions-level journal (TC, TPDS, TCAD or ACM TACO; open, see Section 11). Nothing here claims that bar is reached.

---

2. What is already established (banked evidence)

These results exist and are commit-stamped. They motivate the study; they are not its confirmatory evidence.

Earlier oracle-mask study (L4, A100, H100).

Finding	Evidence and caveat
Kernel speedup does not survive end to end	A100, sparsity 0.75: kernel 1.28x (8192 tokens) and 1.96x (16384) faster than PyTorch's SDPA flash kernel, measured at the model's own head layout. End-to-end prefill with the reference mask builder 0.75x and 0.63x. Flash is not the fastest correct dense kernel on that card: cuDNN SDPA is 25.8% faster at 8192 and FA2 6.0% faster at 16384 (cuDNN faults above 8192). Against them the kernel ratios are 0.95x (a loss) and 1.84x, recomputed from measured per-layer kernel times in the same sweep at the model's (12,2) head layout (`results/s7_sweep_hl122/sweep.parquet`, commit c324515).
The sign depends on the card	32768 tokens, sparsity 0.75: about 1.3x on an L4, about 0.48x on an A100 with the reference builder.
The cause is host-side mask construction	L4 and A100 hosts share a CPU platform; builder costs within about 3%. In the reference builder, about 94% of the builder's loop time (a laptop profile at 32768 tokens, not banked; to be re-measured on the target host under T2.2) is Python overhead in an unvectorised loop, so this is a finding about one implementation. A vectorised builder turned the A100 result into a speedup at 16384: 1.20x against flash, an estimated 1.18x against FA2. At 8192 it reaches 1.02x against flash and an estimated 0.96x (a loss) against cuDNN. The estimates subtract 28 layers times the per-layer kernel difference from the measured dense prefill (`results/s8_vec_endtoend/vec_endtoend.parquet`, commit ae736b4); they are arithmetic, not measurements, and assume the dense prefill is GPU-bound.
The mechanism held on a third card	Of three predictions registered before the H100 run, two passed and one failed (7 of 9 cells).
The oracle is expensive	Scoring costs about 35x the latency the sparsity saves at 32768 on an L4.
A cheap estimator trails the oracle	Mean-pool trails the oracle by 32 to 40 points at 1.5B and 39 to 76 at 7B on multi-key retrieval at 16384.

Pilots on the L4, Qwen2.5-1.5B-Instruct, 1 to 3 October 2026 (called "T4 pilots" after the audit item, not the GPU).

Finding	**Evidence and caveat**
Even the oracle is non-inferior only at density 0.5	Primary claim only on qa_1/16384 (70 vs 68 of 100, bound -9.0); secondary on `niah_multivalue`/16384 (bound -7.7). Nothing at density 0.25 or 0.1.
Inline mean-pool is non-inferior nowhere	7 to 24 points below dense at 0.5; estimator plus mask build cost 0.06 to 0.31 of the kernel saving.
Scalar-threshold XAttention is non-inferior nowhere	At tau = 0.95, bounds -12.7 to -50.8 at densities 0.11 to 0.39; `niah_multivalue`/32768 collapsed to 0 of 50. Dense reproduced 400 of 400.
Density depends on the prompt at fixed tau	33% of blocks on qa_1 vs 11% on NIAH essays at 32768.
Estimator cost depends on the code path	On the L4's torch fallback, XAttention's estimate costs more than dense attention (19.5 vs 14.5 ms at 16384).
Calibrated XAttention	Built and pre-registered; not run (task T1.1).

Standing caveats. Of the A100 figures against the fastest correct dense kernel, the kernel ratios are recomputed from measured kernel times and the end-to-end restatements are arithmetic, not measurements (source: the baseline-strength caveat in `docs/claims.md` dated 3 Oct 2026, on `main` since T0.16); both are to be replaced by same-session measurements (T2.16); at 32768 no faster kernel was measured at the (12,2) layout, so the A100 figures there are against flash only; some 32768-token rows exceed the position limit; results span three mask-rule eras that must not be pooled; the pilots cover one model, one card and synthetic tasks.

---

3. Gaps and nearest work

Gap	**Statement**	**Nearest work (read in full unless marked)**
G1	Learned cross-device predictors exist (CTFusion: 20 training systems, host side is input preprocessing). KernelSight-LM measures host constants per host and assumes they do not depend on the GPU, without testing it. Untested: whether an explicit critical-path model of measured components predicts data-dependent sparse prefill better than a learned predictor when few systems are available.	CTFusion, KernelSight-LM, CloserToMe; nn-Meter, HELP (abstracts only)

Gap	Statement	Nearest work (read in full unless marked)
G2	Host-overhead summaries (TaxBreak's HDBI, SKIP's TKLQT) are ratios or sums. TaxBreak splits host-visible time into framework, library and launch-path parts, and we adapt its null-kernel protocol to measure host costs (null kernels across the whole operation sequence rather than the launch path alone); it does not compose those costs in execution order with synchronisation positions, and the gap between the KernelSight-style step form (our baseline) and the recursion measures what that composition adds. (TaxBreak's remark that host cost is often shown "only as an aggregate residual" describes earlier work, not TaxBreak.) KernelSight-LM (read in full) composes within a step by one formula, GPU time plus dispatch, plus any excess of the host overlap floor over GPU time, plus a serial tail (its Section 5.7, Eq. 4); its discrete-event scheduler advances one event per batch step (Section 4.2), so nothing records where in a step a synchronisation falls, and its synchronisation term is tensor-parallel all-reduce between GPUs. Its Tier B measures host constants on the target with a short serving sweep. Serving simulators (LLMServingSim 2.0, the APEX simulator) work from per-device profiles and model no host work inside a step. TaxBreak says its replay can be imperfect for synchronisation-heavy kernels. Fused GPU kernels can remove host work and synchronisations (VSPrefill merges indices inside its kernel); released estimators often still make them. HiSparse (decode, not prefill) fuses each layer's KV-cache miss handling into one kernel inside the decode CUDA graph, which it says requires no host-side branching; it picks its cache size by profiling sweeps on each platform and finds the best setting depends mainly on host-link bandwidth. It is a system design, not a predictive model. Compilers do not remove data-dependent synchronisations by default: `torch.compile` breaks the graph at `.item()`, `nonzero()` and some forms of tensor `assert` (Section 1); whether its capture options remove the wait on the GPU is measured by the compiled arms. No lightweight model places data-dependent host work and synchronisations inside a prefill step and predicts their cost on an unmeasured pairing.	TaxBreak, SKIP/TKLQT, KernelSight-LM, LLMServingSim 2.0, APEX (Fan et al.), APEX simulator (Lin et al.), VSPrefill, HiSparse
G3	Whether recall is a sufficient accuracy proxy across estimators is untested. The Sparse Frontier lists it as unknown. VSPrefill and Less Is More compare recall across methods, not accuracy at matched recall; VSPrefill's own indexer reaches full-attention accuracy above 90% recall, while Less Is More (decoding) argues that keeping a recency window matters, so prior evidence points both ways. AB-Sparse and CompactAttention compare accuracy at fixed budgets or per-method operating points. Oracle-guided sparse prefill names recall of oracle support as a diagnostic it has not yet run. DFSAttn (video diffusion, not LLM prefill) uses the same recall definition and bounds expected recall from below by the budget times the probability that pooled block scores pick the oracle's top blocks; it compares methods at different sparsities by output similarity, not accuracy at matched recall.	VSPrefill, The Sparse Frontier, Less Is More, oracle-guided sparse prefill, AB-Sparse, CompactAttention, DFSAttn

The full related-work list with read status is in `docs/research_brief.pdf` (the professor brief). Positioning must be re-checked in IEEE Xplore and Scopus before submission (task T7.3).

---

4. Hypotheses

Each has a statistic and a tolerance fixed at lock (marked **[lock]**). Failures are reported as failures. Results are reported per pairing; nothing is pooled into a significance test across pairings.

#	Hypothesis	Test and falsifier
H0	**Separability.** Host components measured with one GPU equal those measured with another GPU on the same host platform; device components measured on one host equal those on another host for the same GPU. Link components are pairing-specific.	Host side: P1 vs P2, and P3 vs P5 vs P6. Device side: P1 vs P3 (L4), P4 vs P6 (H100). Per-operation differences within **[lock]**, after provider-level variance from the dense workload, compared only between pairings that share the arm's operation sequence (Section 5.1); an arm whose sequence differs is reported separately. If H0 fails, H2 is not tested and scope narrows to components measured on the target pairing.

#	Hypothesis	Test and falsifier
H1	**Composition.** On every confirmatory pairing (P2 to P7), the recursion over measured components predicts untraced end-to-end prefill within tolerance, and beats the fitted sum's committed prediction (the primary comparison for G1, trained only on pairings already run, Section 5.4) and also the sum, the KernelSight-style step form and the learned predictor's committed prediction (secondary). P1 is the development pairing: Gate A and the probe bounds are set on it, so its results are reported separately and never counted as a test. Stated in advance: on arms with no synchronisation inside the step (dense, mean-pool, XAttention with its checks removed, and compiled arms with no wait left) the recursion and the KernelSight-style step form should agree; an advantage is predicted only on arms with synchronisations. For the XAttention pair (as shipped, and with its checks removed) the recursion predicts the time difference on each pairing before it is measured. In a CPU check, the plain asserts stayed inside the compiled graph. If they also stay inside it on the GPU pairings, the compiled XAttention pair (as shipped, and with its checks removed) may show only a small gap, timed on those pairings; BUNGA's prediction of that gap on each pairing, made before it is measured, is a direct test of how it prices a wait that compilation leaves in place.	Absolute relative error within **[lock]**; sparse-vs-dense sign correct outside the resolution floor; recursion error below all four baselines' committed predictions on each confirmatory pairing; the retrospective leave-one-pairing-out comparison is reported as secondary; reported separately for dense, synchronisation-free and synchronising arms; the XAttention pair's predicted and measured differences agree within **[lock]** on each pairing. A large advantage on synchronisation-free arms points to a baseline bug, not support for G2. If the learned predictor wins, that is reported.
H2	**Transfer.** Predictions stay within tolerance when components come from other pairings: host components for P2 (from P1) and P5 (from P3); device components for P3 (from P1); both for P6 (host from P3 and P5, device from P4).	Same statistic as H1. P6 is the full-transfer test: only its link microbenchmarks are measured before its prediction is committed. Conditional on H0.
H3	**Forecast.** On P4 (a host platform used nowhere else), each estimator's crossover falls inside the grid interval predicted before P4's workload is timed. The crossover is the bracket between adjacent grid lengths where the sign of sparse minus dense time changes (Section 5.4).	Predicted grid interval vs the measured bracket, widened by the bootstrap interval of the difference at each endpoint. No crossover in range is reported as such.
H4	**Recall sufficiency.** At matched per-layer recall, accuracy does not depend on which estimator chose the blocks, at block sizes 32, 64 and 128. A random-selection control with no inductive bias is held to the same recall: if it matches the estimators, recall is sufficient; if it falls short, which blocks are kept matters.	Paired comparison at matched per-layer recall, or the estimator coefficient in a pre-registered model, against an equivalence bound **[lock]**. Recall is matched only by choosing among each estimator's own budget settings, never by editing masks; the share of rows that find a match is reported. Confirmatory family and multiplicity as in Section 5.7 (Holm); head type and depth thirds are exploratory. The sample size comes from a power rule fixed before lock (Section 5.7); a result below that power is reported as inconclusive. Either outcome is reported.
H5	**Crossover direction (secondary).** (a) On a fixed host, a faster GPU moves the crossover to longer contexts for estimators with host-side steps (AMD: L4, L40S, H100). (b) The shift is smaller for an estimator whose selection runs entirely on the GPU. (c) On a fixed GPU, the host with lower measured host cost gives the shorter crossover.	Signs predicted from measured component ratios before data, with tolerance stated on the input ratio. Consequences of H1, reported separately.

#	Hypothesis	Test and falsifier
H6	**Dispatch.** On held-out pairings (P2 to P7), at 16384 and 32768 tokens (where admissibility is measured), the dispatcher's choice per context length and batch has lower regret than the kernel-speedup rule, with no more decision errors.	Regret is the measured time of the chosen configuration over the best measured admissible configuration (dense included), minus 1; mean and worst case per pairing against **[lock]**. Comparators: the kernel-speedup rule (primary), always dense, always the sparse arm with the best kernel speedup, and the best choice on the nearest measured pairing. If no sparse configuration is admissible and faster anywhere, always-dense has zero regret by construction; the dispatcher can then only tie it, and the test reduces to whether it avoids the kernel-speedup rule's losses. That is reported as such.

---

5. Method

5.1 Definitions

- **Unmeasured pairing.** A host-GPU pairing on which the workload has not yet been timed when the prediction is written. On the target itself the workload runs only in the structural probe (short contexts, under the profiler) and, if the mask gate fails on the first card of a new code path, in an untimed mask-recording pass; every duration from both is discarded. Allowed inputs, stated the same way for every pairing: microbenchmarks on the target (host components measured on that host without the workload on the GPU, and link and synchronisation microbenchmarks on the pairing, including bandwidth under a mask-sized CPU load; seconds); device components of that GPU; and two functional artifacts recorded per code path with every duration discarded: the operation sequence and the estimator's selection (below). No timing or trace of the composed workload on the target is an input: the claim is prediction without timing the workload on the target, from seconds of microbenchmarks there, not zero-shot prediction. The prediction file and its sha256 are synced before the timed workload starts; the analysis refuses any prediction stamped later.
- **Host component** c_host(i): host time to issue operation i (framework dispatch, library front end, launch floor) plus host-side compute such as CPU mask building. Measured by adapting TaxBreak's null-kernel protocol, which applies null kernels to the launch path, to the whole operation sequence (device work replaced by null kernels), and by running host-side compute directly at target sizes. The adaptation is not claimed as a contribution.
- **Device component** c_dev(i): GPU execution time of operation i, from replaying the arm's whole kernel sequence in order on the target GPU, with the masks recorded for its code path, as a CUDA graph (primary; keeps cache state between kernels and has no host gaps). Isolated per-operation replay is the cross-check, and the difference between the two is reported as the cache effect. Never from a run of the composed workload: the replay contains no host, link or synchronisation cost.
- **Link component** c_link(i): pairing-dependent copy time of a non-synchronising operation i (a host-device copy the host does not wait for), from microbenchmarks on the pairing, at the loaded bandwidth where the operation sequence puts the copy alongside host-side estimator work. Synchronisations are costed separately (next item), so no physical event is charged twice.
- **Synchronisation.** Any point where the host waits for the GPU: an explicit synchronise, a copy to the host, or Python reading a GPU value (`.item()`, `bool()` of a tensor, an `assert` on a GPU tensor, sizing from `nonzero()`). Each synchronisation is one operation in the sequence and is costed once, by mechanism m: s_0(m, payload) = s_fixed(m) + payload / B. s_fixed(m) is the host cost of issuing and completing mechanism m (explicit synchronise, `.item()`, `bool()` of a tensor, pinned or pageable copy to the host) with the device idle, measured at the smallest payload so it contains no transfer time. B is the copy bandwidth from the same microbenchmark as non-synchronising copies, loaded where the operation sequence puts the copy alongside host-side estimator work and unloaded otherwise, never

fitted. The wait itself is never a measured constant: the recursion produces it (Section 5.4). For a synchronising operation l_i = d_i = 0. **Composite mechanisms are split:** an `assert` on a GPU tensor and sizing from `nonzero()` each become a kernel operation, with its device cost from replay, followed by a synchronisation; no operation carries both.

- **Operation sequence and synchronisation map.** The ordered operations of each arm, the positions and mechanisms of its GPU-to-CPU synchronisations, and the estimator's selection (its masks on each recorded prompt). Recorded per **code path**: GPU architecture, library versions, and which implementation runs (for example, XAttention's scoring uses Triton on the A100 and H100 and PyTorch elsewhere). A code path already traced is reused (the L4 PyTorch path from P1). A new code path is recorded on the target by a **structural probe**: each arm runs once at short contexts (2048 and 4096 tokens) under the profiler, the order of operations and synchronisations is kept and every duration discarded, and the sequence is extended to the target length by the arm's chunking rule, checked by agreement between the two short contexts. The checker classifies each pairing and arm as "same sequence" or "different sequence"; H0 and H2 compare components only within the same sequence. The **estimator's selection** is recorded per code path at the grid lengths, untimed, since density depends on length and the short probe cannot give it. P1's masks serve the L4 PyTorch path. On P2 both XAttention code paths run on the same card through the existing `use_triton` argument (the mask gate, Section 7); masks are recorded on the target only for the first card of a new code path, and only if that gate fails. P4's recorded Triton masks are compared with P2's: if they differ, P6 records its own masks and the full-transfer claim (H2) is restated.
- **Recall.** At a layer, the share of that layer's dense attention mass, computed from its actual inputs in the arm's own forward pass, that the kept blocks capture (primary). Recall against a separate dense pass (whose scores are already cached) is a secondary measure. H4 is read on the primary one.
- **Accuracy.** Paired non-inferiority against dense (test in Section 5.7).
- **Admissible configuration.** Dense, or an arm at a density whose accuracy is non-inferior to dense (Section 5.7). Admissibility comes from accuracy measured once on the L4 (the primary GPU, Section 5.7), or from recall if H4 holds; it is not re-measured per pairing. It is a decision input, not a confirmatory claim (Section 5.7). It transfers to a pairing on another code path only if the mask gate passes (Section 7); otherwise the dispatcher is scored only on pairings that share the L4's code path, and that restriction is reported.
- **Kernel-speedup rule.** The decision a practitioner would make from published results: use an estimator at its admissible density wherever its sparse kernel alone beats the selected dense kernel on that GPU, ignoring estimator, host and synchronisation costs. Kernel times are device components, so the rule uses no input the model lacks.
- **Profitable.** End-to-end speedup above 1 over the fastest correct dense kernel, at non-inferior accuracy, estimator cost included. **Predicted profitability** on an unmeasured pairing combines the time model with the admissible density per estimator: from recall if H4 holds, otherwise from that estimator's accuracy runs.

5.2 Hardware pairings

Pairing	Host platform	GPU	Provider	Role
P1	Intel Cascade Lake	L4	GCP G2	Development pairing (not a confirmatory test of H1); source of transferred components
P2	Intel Cascade Lake	A100	GCP A2	Host-side H0; host transfer (H2)
P4	Intel Sapphire Rapids	H100	GCP A3	Held-out host platform: forecast (H3)
P3	AMD EPYC 3rd gen	L4	AWS G6	Device-side H0; device transfer (H2)
P5	AMD EPYC 3rd gen	L40S	AWS G6e	Host-side H0; host transfer; GPU-speed axis (H5a)
P6	AMD EPYC 3rd gen (7R13)	H100	AWS p5.4xlarge	Device-side H0; full transfer (H2); H5a, H5c. Run last
P7 (optional)	AMD EPYC 2nd gen	A10G	AWS G5	Extra held-out point if time allows

Run order: P1, P2, P4, then P3, P5, P6 and P7. Every transferred component still exists before it is used: P4 needs only its own components and P1's operation sequence, and P6 needs P3, P4 and P5. The GCP pairings go first because GCP sessions have a last launch date (Section 6, rule 14) and the AWS tooling (T5.1) is the likeliest delay.

Pinned GCP machines and zones. The machine series fixes the host platform.

Pairing	Machine type	Zone	Why this zone
P1	`g2-standard-8` + 1 L4	`asia-northeast1-a` (fallback `asia-northeast1-c`)	Used by the pilots; the plan's L4 zone
P2	`a2-ultragpu-1g` (A100 80GB)	`asia-southeast1-c`	The only zone offering this machine when re-verified on 16 Sep 2026; same machine as the earlier A100 evidence
P4	`a3-highgpu-1g` (H100 80GB, DWS flex-start)	`us-central1-a` (fallback `asia-southeast1`)	Launch-script default; both regions priced on file

Never an Australia region. Availability and rates are re-checked from the researcher's Mac before each session.

Rates on 20 Oct 2026 (T0.17). Per hour from the Cloud Billing catalogue. The USD/h and ₹/h columns are the GPU, plus vCPUs times the core rate, plus GiB of RAM times the RAM rate, with no disks. The planning rate, all-in, adds the 200 GB balanced boot disk that each launch script creates and the local SSD that the machine type bundles: one partition for `a2-ultragpu-1g`, two for `a3-highgpu-1g`, none for `g2-standard-8`. Disks are priced per GiB-month at the SKU for the region and provisioning model: balanced PD `16D2-FC49-A127` (`asia-southeast1`), `6AE1-525F-8B80` (`us-central1`) and `6D8C-DF09-314D` (`asia-northeast1`); local SSD `80AE-FFEA-2D34` (flex-start, `asia-southeast1`), `9F7C-3C75-F39B` (Spot, `asia-southeast1`) and `2303-1D6A-9C08` (flex-start, `us-central1`). Two inputs are assumptions, not read from the catalogue: 730 hours per month, and 375 GiB per local-SSD partition. Rupee figures are the catalogue's INR prices (₹95.99 per US dollar).

Pairing	Model	Region	GPU SKU	Core SKU	RAM SKU	USD/h	₹/h	Planning rate, all-in, ₹/h (USD/h)
P2 (A100)	Flex-start	`asia-southeast1`	`293E-F9AA-0457`	`416B-7055-D3FE`	`B392-5C03-B893`	3.0236	290.23	297.47 (3.0990)
P2 (A100)	Spot	`asia-southeast1`	`36BB-6DC9-2245`	`960A-A3F4-7AD1`	`7AF9-D5F3-1235`	3.5923	344.82	350.14 (3.6477)
P4 (H100)	Flex-start	`us-central1`	`341A-49A5-0C07`	`9A32-36B2-7FBC`	`ED19-A584-4A84`	4.7078	451.90	462.42 (4.8174)
P4 (H100)	Flex-start	`asia-southeast1`	`7CF8-F592-DE2C`	`4529-B360-744E`	`EF4B-260A-DD01`	4.7078	451.90	463.47 (4.8284)
P1 (L4)	On-demand	`asia-northeast1`	`FDE8-86D4-B581`	`ECDA-3E53-7A9E`	`68F1-6598-F513`	1.0962	105.23	108.64 (1.1318)

Each session's cost cap (rule 10), and the rate given to the teardown script from which `docs/spend_ledger.md` is updated (rule 11), use the planning rate, all-in. On 20 Oct 2026 two places still hold September figures: the teardown script's default rate (₹80 per hour, the L4's) and the frontier budget gate's rate table on the `t11-run` branch (₹284, ₹321, ₹425 and ₹80 per hour).

Funding (researcher's decision, 21 Oct 2026). The GCP credit expires on 1 Dec 2026. New spend through 30 Nov 2026 23:59 IST has one budget of ₹14,000, counted at the planning rates above. An expense after that is approved by the researcher personally and paid outside the credit; the launch scripts say so when they run after 30 Nov 2026 (India time). Per-session caps (rule 10) are unchanged. Tasks without a planned duration (T3.1, T3.2, T6.1 and T6.4) are priced when they are scheduled, and any that do not fit before 30 Nov 2026 run after it. The last GCP launch date is 9 Dec 2026 (India time, inclusive; rule 14).

"Same platform" is not the same SKU: CPU model string, core count, SMT state, observed frequency, CPU microcode version, the provider's instance generation and the provisioning model (on-demand, spot, DWS flex-start or reserved) are recorded on every row (providers change hardware without notice) and compared before H0 is read.

5.3 Per-pairing session protocol

1. **Environment.** One container image across providers; driver matched where possible and recorded; threads pinned to the GPU's NUMA node; CPU frequency sampled; clocks locked where permitted and recorded; GPU clocks and throttle reasons sampled during every timed block.
2. **Host-speed probe** (fixed single-thread loop plus null-kernel launch burst) at the start, middle and end of the session and between ground-truth blocks.
3. **Link microbenchmarks.** Bandwidth both ways, pinned and pageable; s_fixed per synchronisation mechanism at the smallest payload, with the device idle (Section 5.1); PCIe generation and width; bandwidth under a concurrent CPU load sized like the largest mask build of any arm at each context length; launch-queue depth Q. Then the **structural probe** for any arm whose code path has not been traced (Section 5.1); durations discarded.
4. **Host components.** Operation sequence with null kernels; CPU-side compute at target sizes.
5. **Device components.** Every arm's kernel sequence replayed in order as a CUDA graph (masks and shapes recorded per code path, Section 5.1; data-dependent work replayed with each prompt's recorded mask). Every operation also replayed in isolation as a cross-check.
6. **Prediction committed.** Computed from steps 3 to 5 (plus transferred components for H2), written with its sha256 and synced.
7. **Ground truth.** Untraced end-to-end prefill per arm, density (a set fixed at lock that includes every admissible density, so every configuration the dispatcher can choose is measured), context length and batch, arms interleaved, with a dense control. Median, p90 and p99 over repeats are reported. Mask density and executed density are recorded on every row. **Context grid:** 2048, 4096, 8192, 16384 and 32768 tokens for every arm, balanced across arms; nothing above 32768, since the Qwen2.5 models stop there and Llama-3.1-8B does not fit at 65536 on an L4. Each cell is checked at lock against the model's position limit (for accuracy cells, prompt plus generated tokens). 2048 and 4096 are **model-validation points, not operating points**: they test the model where host work and synchronisations are the largest share of runtime, and check the structural probe's extrapolation against measured time. No profitability claim is made there, and H6 is scored only at 16384 and 32768. Timed short lengths cost like any other cell, since the structural probe discards its durations. The cell count is controlled on the arm-and-density cross, not on length: compiled variants that T2.21's graph-break report shows identical to another variant are not timed separately; the precomputed-mask ablation runs only at admissible densities; window and dense each run at their single density. If the budget still forces cutting lengths, the full grid stays with the XAttention pair, the compiled XAttention pair, mean-pool and dense, and other arms take 8192, 16384 and 32768.
8. **Profiler overhead.** Traced vs untraced end-to-end time per arm.

5.4 Cost model

For one CUDA stream, operation i has host cost h_i = c_host(i), device cost d_i = c_dev(i) (GPU execution only) and link cost l_i = c_link(i) (copy time; zero for a kernel). H_i is when the host has issued operation i; D_i is when the device finishes it.

```
H_i = H_(i-1) + h_i
if Q launches are pending:   H_i = max(H_i, D_(i-Q))          (finite launch queue, Q measured per pairing)
if operation i is a synchronisation:  H_i = max(H_i, D_(i-1)) + s_0(m_i, payload_i);  D_i = H_i (l_i = d_i = 0)
otherwise:  D_i = max(H_i, D_(i-1)) + l_i + d_i                  (l_i at the loaded bandwidth where the operation sequence puts a copy alongside host-side estimator work)
end-to-end prefill time = D_N
```

Only device costs transfer between hosts (H2); link and synchronisation costs are always measured on the target pairing. Copies and kernels on one stream are serialised. **Scope:** "one stream" means the GPU executes its own work in order; host and GPU still run asynchronously, and the recursion models that overlap (the host runs up to Q launches ahead of the device, and waits only at synchronisations). Every arm here issues its GPU work on one CUDA stream, which T2.3 checks from the stream IDs in the traces. Claims do not extend to implementations that spread one forward pass over several GPU streams. Contention is costed from the loaded-bandwidth microbenchmark on the target, never fitted and never taken from the workload. It inflates only transfer terms (l_i, and the payload term of s_0), never host costs, so host-memory pressure is charged once. Each pairing's ground-truth trace is checked afterwards for copy-kernel concurrency and unified-memory faults, as a check on the model, never as an input.

Baselines from the same components.

- **Sum:** sum(h) + sum(l) + sum(d), no overlap.
- **Fitted sum:** a·sum(h) + b·sum(l) + c·sum(d), with a, b, c ≥ 0 fitted on the same rows as the learned baseline (see Fitting the baselines below). It uses the recursion's inputs with minimal fitting and no structure, so it is the fair test of G1 and the primary comparison: beating only LightGBM, which trains on at most six pairings, would not show that the structure helps.
- **KernelSight-style step form:** max(sum h, sum l + sum d) per layer, plus every synchronisation cost as a serial tail. It differs from the recursion only through operation order, synchronisation positions and the queue term, so the gap between the two isolates G2.
- **Learned (CTFusion-style LightGBM):** task features (context length, density, batch, operation and synchronisation counts), hardware microbenchmark features (host, link, device costs; pinned and pageable bandwidth) and bottleneck ratios (arithmetic intensity per operation class, host-to-device time ratio, transfer demand relative to link bandwidth, synchronisations per operation); trained as in Fitting the baselines below; features and hyperparameters fixed at lock. Tests whether the structure adds anything over regression (G1; secondary comparison). A learning curve reports its error when trained on 1 to 6 pairings, to show whether it is short of data or simply worse.
- **HDBI** reported as a diagnostic only.
- **Constant host cost:** the recursion with every host cost replaced by one per-host constant (the median null-kernel launch cost). Tests whether per-operation host costs matter.
- **Nearest measured pairing:** the measured end-to-end time of the most similar pairing already run (same GPU if one exists, otherwise same host platform), unscaled. A naive baseline the model must beat to be worth building.

Fitting the baselines. The fitted sum and the learned predictor make committed predictions the same way the recursion does. Each prediction for a pairing is trained only on pairings whose ground truth was banked before that prediction is written, and it is hashed and synced with the recursion's prediction before the workload is timed (K9, T2.11). In the planned run order P2 is predicted from P1 alone and P4 from P1 and P2. The training set is whatever has been banked when the prediction is committed, and each prediction file records it. The learned predictor's committed prediction for P2 is trained on one pairing and sees no variation in its hardware features; its learning curve (T2.10) and the retrospective fits show how much this matters. These committed predictions are the ones H1 scores. Full leave-one-pairing-out fits, trained on every other pairing including those run later, are reported as a retrospective secondary analysis, labelled as such, because a forecaster could not have had the later pairings. Gate A (T3.3) stays at the sum and the KernelSight-style step form, because on P1 no other pairing exists to train a fitted baseline.

Reported for every prediction: absolute relative error; decision error (predicted profitable when it is not, and the reverse); crossover-length error; and a prediction interval from bootstrapping the component measurements. Errors inside the noise floor (Section 5.6) are not interpreted.

Model ablations (from the same components, no extra runs): the recursion without the synchronisation term; host-only (sum h) and device-only (sum l + sum d) lower bounds; the queue sensitivity below.

Batch. Batch 1 and 4 at 16384 on the L4 pairings; batch 4 at 32768 where memory allows on the A100 and H100. **Crossover.** Sparse is profitable where D_N(sparse) < D_N(dense). The crossover is reported as the bracket between adjacent grid lengths where the sign of sparse minus dense time changes, widened by the bootstrap interval of the difference at each endpoint; the same rule is applied to predicted and to measured times. A quadratic fit a·L^2 + b·L + c per arm is a secondary point estimate, reported with its residual (a form that fits dense well and sparse poorly is itself a finding). No crossover in range is reported as such, a location inside the resolution floor is not interpreted, and two crossovers are possible. **Queue sensitivity.** Every committed prediction is recomputed with Q at 0.25x, 0.5x, 2x and 4x the measured value, and with no queue term. **Density sensitivity.** Every committed prediction is also recomputed at plus and minus the measured cross-path density difference (the mask gate, Section 7).

5.5 Arms

Arm	**What it does**	**Mask built on**	**Synchronisations per layer**	**Role**
Dense	Fastest correct kernel per card, context length and batch, chosen by the protocol below	No mask	None expected (checked in the trace, T2.3)	Reference
Oracle	Pools full dense-softmax attention mass to blocks, from a separate dense pass (one ranking per KV head, meaned over its query heads)	GPU (separate dense pass)	Recorded in T2.3	Attention-mass reference; highest cost. **Not** a guaranteed quality ceiling
Window	Sink plus local blocks	Fixed pattern	None expected	Zero-cost floor
Mean-pool	Pooled query and key blocks (MInference-style); inline device builder	GPU	None (measured; `docs/claims.md`)	Cheap deployable baseline
Vertical-slash style	Vertical and slash scoring reduced to blocks	GPU (to be built, T1.7)	Recorded in T2.3	Different estimator structure
Calibrated XAttention	Antidiagonal scoring with per-(layer, head) thresholds from the authors' profiler, run as shipped. XAttention: Xu et al., "XAttention: Block Sparse Attention with Antidiagonal Scoring", ICML 2025; arXiv:2503.16428; code `mit-han-lab/x-attention`, used at commit e379887, installed and called, with the lines listed in `docs/vendor_register.md` reproduced	GPU (Triton on A100 and H100, PyTorch elsewhere)	At least two per query chunk: `assert` on GPU tensors in `find_blocks_chunked` (upstream commit e379887)	Data-dependent density and synchronisations
XAttention, Python asserts removed	The authors' unmodified `xattn` package loaded through a module-scoped import hook that compiles only that package with `optimize=1`, which strips its Python `assert` statements and `if __debug__` blocks. Nothing is copied (rule 8), and the rest of the process, including the block-sparse package shared with mean-pool, runs unchanged. The hook compiles from source and never reads or writes the bytecode cache, so stripped bytecode cannot reach the default arm. Every session verifies that the default arm still carries its asserts and that the stripped code differs only in assert sequences and `__debug__` blocks. Triton device asserts, `TORCH_CHECK` and other C-level checks stay in place, hence the name. Whole-process `python -O` is not used (Appendix A, 12 Oct 2026). Accepted only if its masks and outputs are bitwise identical to the shipped arm on every recorded prompt	GPU	The Python assert waits removed; any left are recorded in T2.3	Paired test of G2: same estimator and kernel, with and without synchronisations

Arm	What it does	Mask built on	Synchronisations per layer	Role
Compiled variants	Dense, mean-pool, vertical-slash and both XAttention arms wrapped in `torch.compile` (mode `reduce-overhead`, which uses CUDA graphs where the graph allows), with default settings and again with the capture options for scalar outputs and data-dependent shapes. The authors' code is wrapped, not edited (rule 8). Compile time and warm-up are excluded from timed runs and reported separately. Masks are compared with the eager arm on every recorded prompt; a variant whose masks differ on any prompt is reported as a different arm, with the agreement share	As the eager arm	Graph breaks per layer recorded on CPU (T2.21); waits that remain recorded on the GPU (T2.3)	Tests whether compilation removes the synchronisations G2 prices. Each compiled variant is its own code path (K8), never pooled with eager
DuoAttention-style (conditional)	Retrieval heads dense, streaming heads sink plus local, from released head patterns; only if patterns exist for a study model and the licence allows	Fixed per head	None expected	Head-differentiated recall profile for H4
Precomputed-mask (ablation)	Each estimator's masks computed offline and preloaded; then the same kernel	Preloaded	None	Separates estimator compute and synchronisation from kernel and launch cost
Random at matched recall (H4 control)	For each layer, blocks drawn uniformly at random (fixed, recorded seeds) and added until the layer's recall reaches the target, with the same mandatory blocks as every arm; untimed	Offline	Not timed	No inductive bias: tests whether recall alone carries accuracy. Its density is reported, since it will need more blocks

All arms use one per-head selector and one block-sparse kernel. A bitwise gate requires the selector to reproduce the head-uniform rule exactly when heads share scores. Timed block size is 128. Every row records mask density, executed density and `sys.flags.optimize` (under K8 an interpreter flag is a distinct code path).

Dense-baseline selection (control K2). Candidates: SDPA flash, SDPA cuDNN, SDPA memory-efficient, FA2, and FA3 on the H100. A candidate is *correct* if it matches the float64 naive reference of `attnbench/gates.py` on fixed inputs within a tolerance fixed at lock, or, where that reference cannot be allocated, passes the gate's cross-backend comparison (weaker evidence), and runs N repeats without a device fault. Which of the two each cell used is named per cell (cuDNN faulted above 8192 tokens on the L4 and A100). The *fastest* correct candidate has the lowest median untraced time per card, context length and batch, measured in a warm-up block separate from ground truth. The choice is recorded before the timed runs, and every speedup is stated against it. Banked A100 figures are restated against it (Section 2).

5.6 Confounders and controls

Confounder	Control
CPU microarchitecture, caches, memory bandwidth	Inside c_host, measured on the target host at target sizes; never synthesised
PCIe generation, copy and sync latency	c_link measured per pairing; generation and width recorded

Confounder	**Control**
Cloud CPU frequency (governor and true clock often hidden)	Host-speed probe distributions per session; a session is usable only if the probe's coefficient of variation is under a bound; a component transfers only if source and target probe medians differ by less than a bound. Both bounds fixed at lock from P1 repeat sessions. Failing sessions are rerun
vCPU topology, SMT, noisy neighbours	Pinning; topology and tenancy recorded; probe detects drift; P1 repeat sessions give the between-session floor
Host-memory contention between mask building and DMA	Link bandwidth also measured under mask-sized CPU load; copies in flight during host-side estimator work are costed at the loaded bandwidth
Driver and software differences	One container image; driver recorded. H0 compares components only between sessions with the same driver version; where versions differ, the comparison is reported as confounded. Dense control every session
Operation sequence differs by GPU or library	Sequence recorded per code path (Section 5.1); H0 and H2 only within the same sequence
Cloud noise floor	Between-session spread of end-to-end time from the P1 repeat sessions (T3.2), and each pairing's within-session spread, reported as numbers; prediction errors inside them are not interpreted. A bare-metal control is out of budget and the claims concern cloud hosts; this is stated as a limitation
Dense kernel choice	Selection protocol (K2, Section 5.5); speedups stated only against the selected kernel. The selected kernel's time in every timed block is compared with its warm-up time; a drift beyond the resolution floor flags the block
GPU cache state between kernels	Device time from in-order graph replay; isolated replay as the cross-check, difference reported
GPU clock throttling	Clocks and throttle reasons sampled in every timed block; throttled blocks flagged and rerun
Measurement asymmetry between arms	One device-time method (in-order graph replay) for all arms; isolated replay only as a cross-check
Prediction made trivial	No prediction input comes from the composed workload on the predicted pairing
Profiler overhead	Ground truth from untraced runs
Compilation	Compile time, warm-up and recompiles excluded from timed runs; recompile count recorded per block, and a recompile inside a timed block flags it
Predictions fitted after the fact (K9)	Predictions hashed and synced before the workload runs
NUMA placement	Threads pinned to the GPU's local node; microbenchmarks from that node
Pinned vs pageable memory	Both measured; each traced copy classified and costed
Launch-queue backpressure	Q measured per pairing; host costs measured in bursts
Unified memory	Not used; any fault flags the row
Code-path mask divergence (XAttention scores with Triton on the A100 and H100, PyTorch elsewhere)	Mask gate on P2 (both paths on one card through `use_triton`) and the P4 against P2 Triton comparison (Section 7); per-layer mask agreement and density difference reported; density sensitivity on every prediction (Section 5.4); admissibility transfers across code paths only if the gate passes (Section 5.1)
Interpreter flags	`sys.flags.optimize` recorded on every row; a different flag is a different code path (K8)
Replay mismatch	Replayed kernel names and durations compared with the P1 trace per operation family before H1 is read; mismatched variants flagged
Null-kernel floor plausibility	P4's floor reported with its exact definition and compared only with a published figure measured the same way (TaxBreak about 4.7 us on Xeon 8480C with H100; SKIP 2.37 us on Xeon 8468V with H100 PCIe)

Study controls K1 to K9. K1 one selector and kernel for every arm, Partly built (head-uniform selector and kernel built; per-head selector built, on `main` since T0.16); K2 fastest correct dense baseline per card, context length and batch (protocol in Section 5.5; not built); K3 disjoint calibration splits with a pre-run gate (built); K4 three models, Qwen2.5-1.5B, Qwen2.5-7B, Llama-3.1-8B held out of all fitting (planned); K5 recall at block sizes 16 to 128 at matched density (control only; AB-Sparse already reports the decode-side version); K6 pre-registration, checks that fail on purpose, provenance on every row (in place); K7 cells count only if dense accuracy clears a floor (not built); K8 code path recorded per row, fallbacks never on the same curve (built); K9 predictions hashed before the workload (not built).

5.7 Quality measures

- **Accuracy test (pre-registered):** exact paired non-inferiority, one-sided alpha 0.025, as used in the pilots: a union bound over one-sided Clopper-Pearson bounds on the two discordant cells (`attnbench/analysis/exact_noninferiority.py`). The dense arm must reproduce exactly in every run on the same card.
- **Primary accuracy statement:** the achieved one-sided lower confidence bound on the paired accuracy difference, in points, reported continuously. The margin is used only to produce the admissibility label the dispatcher consumes: 10 points primary, with sensitivity at 5 points. No 2-point column: with no discordant pairs the union bound's floor is 4.3 points at $n = 100$ and 2.2 at $n = 200$, and about 220 examples are needed for 2 points. Absolute and relative differences are both reported (K7).
- **Margin against speedup:** for each margin and configuration, whether it is admissible and the speedup it buys, reported as a frontier from the same accuracy rows (no GPU cost).
- **Primary cell size:** `qa_1`/16384 is sized once at $n = \max(200, \text{the H4 power rule})$, so the same rows serve admissibility and H4. At 200 the 5-point label is reachable at moderate discordance. Secondary cells keep the 10-point margin, stated as such. Admissibility n is fixed at lock.
- **Admissibility family:** admissibility is a decision input, not a confirmatory claim. Its Holm family is declared at lock as every admissibility test that feeds the dispatcher. Correcting shrinks the admissible set, which can move regret either way, so dispatcher regret is reported with and without the correction.
- **Replacement test:** a more powerful paired test is adopted only if simulation shows coverage at or above nominal across dense accuracies including the near-ceiling region and across the pilots' discordance rates, with power secondary among tests that pass coverage. The choice is recorded in the lock commit. Candidates are leads only until read in full (rule 5).
- **Recall:** both references (Section 5.1), per layer, on the very masks that produced each accuracy row; also by head type (retrieval-like vs local, criterion fixed at lock) and by depth third.
- **Intrinsic recall:** budgets 0.50, 0.25, 0.10, 0.05; null scorer as a floor; budget selection on 32 examples per task.
- **H4 across block sizes:** on the primary cell (`qa_1`/16384; n as in the primary cell size above), block sizes 32, 64 and 128 through an untimed reference masked-attention path; block 16 only if that path supports it.
- **Recall matching:** an estimator reaches a target recall only through its own budget settings (threshold, top-k or density), never by editing its masks. The share of rows that find a match is reported.
- **H4 equivalence bound:** justified at lock from the tasks (how large an accuracy difference would change which estimator a practitioner picks). The default is 10 points; T1.5 also prices a 5-point bound, which needs about four times the examples.
- **H4 power rule (fixed at lock):** $n \approx (z(1-\alpha) + z(1-\beta/2))^2 \times p / \delta^2$, where p is the rate at which two estimators disagree on an example (measured on the banked pilot rows, T1.5), δ is the equivalence bound, α is the one-sided level for each of the two tests and $1-\beta$ is the power, all fixed at lock, with α adjusted by Holm across the confirmatory family. p is the upper 95% confidence bound of the pilot disagreement rate, not its point estimate. At $\alpha = 0.025$, 90% power and $\delta = 10$ points this is about 1,300 × p (130 at 10% disagreement, 260 at 20%). Bonferroni over three comparisons gives about 1,630 × p, and Holm needs no more than Bonferroni; the family size, and so n, is fixed at lock. The exact test needs more.
- **H4 confirmatory family:** the planned estimator pairs at block sizes 32, 64 and 128 on the primary cell, plus the random-selection control against each estimator, Holm-corrected together. Head type and depth thirds are exploratory and spend no α. An H4 result below the fixed power is reported as inconclusive, never as support or refutation.

- **Where measured:** once, on the L4 (the primary GPU), with the dense control repeated on the A100. The cross-card control's acceptance criterion is a task-score bound fixed at lock, with the exact-match rate reported alongside; bitwise reproduction across cards is not expected.
- **Dense floor (K7)** and **critical density** (lowest non-inferior oracle density; descriptive only).

5.8 Extension: chunked prefill

On P5, with inline mean-pool and calibrated XAttention, prefill also runs in chunks of 2048 and 4096 tokens at 16384 and 32768. Each chunk reruns the estimator over the accumulated keys, which the recursion represents as repeated operations; the predicted re-selection overhead is committed first. First check whether the block-sparse kernel supports query chunks shorter than the key length; if not, drop the extension and record it. The kernel's time at each query-to-key ratio is recorded separately from the per-chunk estimator operations, and mask density separately from executed density. This reruns the estimator per chunk and is not presented as state-of-the-art chunked-prefill engineering. External reference only: CompactAttention reports XAttention at 0.68x to 0.84x of dense speed end to end under chunked prefill on two GPUs.

5.9 Application: profitability dispatcher

What it decides. For each pairing, context length and batch, one configuration: dense, or a sparse arm (eager or compiled) at one of its admissible densities. It picks the admissible configuration with the shortest predicted end-to-end time, and dense unless a sparse configuration is predicted faster by more than the resolution floor.

Inputs. Only what a prediction may use on an unmeasured pairing (Section 5.1), plus admissibility. The choice is written into the same hashed prediction file as the time prediction, before the workload runs (K9).

Scoring. On each held-out pairing, from the measured ground-truth grid (Section 5.3, step 7), so it needs no extra GPU sessions: regret (H6), decision errors (sparse chosen where dense was faster, and the reverse), and time gained or lost against always-dense.

Comparators. The kernel-speedup rule (Section 5.1; primary), always dense, always the sparse arm with the best kernel speedup, and the best measured choice on the nearest measured pairing.

Scope. An offline decision per pairing, context length and batch, not a runtime scheduler that reacts to live load (Section 10).

---

6. Operating rules (both Claudes, every session)

Claims and evidence

1. Never claim "no accuracy loss" or "zero accuracy cost". Report non-inferiority at a stated margin.
2. Keep oracle, deployable-estimator, CPU-builder and GPU-builder results distinct; never put them on one curve.
3. Every measurement is commit-stamped with `git_dirty = False`; results from different mask-rule eras or code paths are never pooled.
4. Never delete or rewrite historical audit findings. Withdrawn or corrected items are annotated in place.
5. Cite only papers read in full; anything else is marked "from abstract". If a paper is needed and not available, ask the researcher for the PDF instead of characterising it from search summaries.
6. Predictions are written and hashed before the data they predict.

Code

7. Run the focused tests for anything changed before committing; keep the full suite green.
8. Do not commit unrelated changes. Third-party code is installed and called, never copied, except as listed in `docs/vendor_register.md`. Each entry names the upstream, the commit, the licence, which lines are verbatim or adapted, the licence and notice that travel with it, why it cannot be installed and

called, and the date the researcher approved it. Code from a repository with no licence is never vendored. Its lines are reproduced only where `docs/vendor_register.md` lists them and the researcher's approval is recorded in `docs/permission_log.md`.

9. No model identifiers or assistant attribution in research artifacts beyond the commit trailers the tooling adds.

Cloud sessions (GCP now, AWS later)

10. Before any GPU session, state the exact command, estimated cost, cost cap, hard-delete limit and artifact path, and wait for approval. The researcher runs all cloud commands from their Mac.
11. Launch sequence: cleanup check, cost cap, hard-delete limit, stale-results quarantine, deploy the clean commit, preflight, run, teardown through the teardown script (which writes `session_cost.txt`). Update `docs/spend_ledger.md` from that file.
12. L4 zone `asia-northeast1-a` (fallback zones as in the launch scripts). Never an Australia region.
13. AWS sessions only after the AWS tooling meets the same guarantees and passes tests against a fake CLI (task T5.1).
14. No GCP session launches after **9 December 2026** (India time; the date itself is allowed). The launch scripts refuse later launches (`scripts/_launch_policy.sh`). GCP work (T1.1, T3.1, T3.2, T6.1, T6.4) is planned to finish before then.

Permissions

15. Any use of results that needs the researcher's permission is recorded in `docs/permission_log.md` before the use. A use not in the log is not approved.

Writing for the professor: plain language, short sentences, no em dashes.

7. Decision gates

Gate	Condition	If it fails
Gate A: P1	The recursion beats the sum and the KernelSight-style step form on P1 (T3.3); P1's replay-versus-trace difference per operation family is within a bound fixed at lock; and the delay-injection slope check passes: each synchronisation mechanism is injected separately into the dense arm on the L4, where the GPU is busy and where it is idle, and the ratio of predicted to measured time per injected synchronisation is within a bound of 1.0 fixed at lock, one slope per mechanism (double counting gives about 2; an instrument check, Section 10). The replay-versus-trace check reruns on the first card of every new code path before H1 is read there	Stop; revise the model, the synchronisation costing, or the device-cost method if the replay check failed, before any new pairing
Mask gate: P2	On P2, XAttention's Triton and PyTorch paths run on the same recorded prompts through `use_triton`; per-layer mask agreement and density difference against P1's recorded masks are within a bound fixed at lock. P4's Triton masks are then compared with P2's	Masks are recorded on the target, untimed, for the first card of the new code path; admissibility does not transfer to that code path (Section 5.1). If P4 and P2 differ, P6 records its own masks and the full-transfer claim is restated
Null-kernel floor	P4's null-kernel launch floor is within a fixed multiple (set at lock) of the published figures measured the same way (Section 5.6)	Investigate and report before H3 is read; H3 is read only with the cause stated
Gate B: lock	All **[lock]** values fixed, two adversarial reads done, lock commit tagged (T4.x)	No confirmatory pairing runs
Gate C: AWS	AWS tooling passes fake-CLI tests; quotas granted; each pairing costed	No AWS launch
Gate D: H0	Separability holds	H2 not tested; scope narrows to per-pairing composition

Gate	Condition	If it fails
Last GCP launch	Every GCP session launches by 9 Dec 2026	No GCP launch after the date; re-plan the remaining GCP work with the researcher
Budget	Each session within its cap	Cut in an order fixed at lock (proposed: P7 first, then the chunked extension); the cut order never removes ground-truth timing, H1 or H4

---

8. Task list

Owner: **Code** = Claude Code, **Chat** = Claude chat with the researcher, **R** = researcher. IDs are permanent; do not renumber. Add new tasks at the end of a phase.

Phase 0: Repository transition

ID	Task	Owner	Depends	Done when	Status
T0.1	Commit this plan, its PDF, `CLAUDE.md` and the build tool	Code		On `main`	Done (merged to `main` 4 Oct 2026)
T0.2	Brief Claude chat: upload the PDF to the project, remove superseded plan PDFs, paste the handover note	R	T0.1	Chat answers a test question from the plan correctly	Done (4 Oct 2026)
T0.3	Tag the pre-pivot state (`pre-pivot-2026-10-04`) so every banked result stays reproducible	Code (R approves push)		Tag on origin	Done (18 Oct 2026): `pre-pivot-2026-10-04` confirmed on origin at `7bf2a01` (`git ls-remote --tags`)
T0.4	Review `docs/code_inventory.md`: confirm keep / adapt / archive for every module and script	R + Chat decide; Code verifies by reading the code	T0.1	Every row has a confirmed decision	Done (4 Oct 2026; "Check" rows are resolved by reading the code during T0.5)
T0.5	Move archived code and its tests to `legacy/` (not deleted), fix imports, keep the suite green	Code	T0.3, T0.4	Suite green; no live module imports `legacy/`	Not started
T0.6	Move superseded docs to `docs/archive/` unchanged; update test references to their paths	Code	T0.3	Suite green	Not started
T0.7	Rewrite `README.md` for the new question; remove claims that break rule 1	Code	T0.1	README matches Sections 1 and 2	Done (19 Oct 2026): README on main; rewritten for public readers
T0.8	Point `.github/copilot-instructions.md` at this plan	Code	T0.1	Done	Not started

ID	Task	Owner	Depends	Done when	Status
T0.9	Carried-over housekeeping: `ledger-updates` work at `dc707e3` (cherry-pick of the local-only `b00019f`), spend ledger brought up to date, audit addenda	Code		Confirmed with R and done	Decision (scope to confirm). On 18 Oct 2026 the sessions table in `docs/spend_ledger.md` had no row for 15 sessions that have a `session_cost.txt`: the seven Stage 2 segments of 4 Sep 2026 (one now under `results/_discarded/`), segment 3 of 5 Sep, Stage 5b of 7 Sep, and six T4 sessions of 1 and 2 Oct (dense probe, selected pilot, sparse canary, 32768 sparse pilot, XAttention canary, XAttention pilot). The 32768 pilot's cost file is pasted below the table, and `dc707e3` (cherry-pick of the local-only `b00019f`) pastes the two XAttention ones; none is a table row. The 2 Oct 16384 sparse pilot session has no cost file
T0.10	Archive the last Estimator Frontier plan (with referee responses) and the professor brief in `docs/archive/` and `docs/`	Code	T0.1	Files committed	Done
T0.11	Bring the GCP launch scripts in line with the plan: pinned default zones, refusal of Australia regions and of launches after the last GCP launch date; re-check rates	Code		Tests pass; rates confirmed by R	Partly built (zones and refusals done and tested; rate re-check not done)
T0.12	Find the source of the caveat "some A100 dense baselines were estimated"; keep it with its source, or drop it with a dated note	Code		Recorded here	Done (5 Oct 2026): the baseline-strength caveat in `docs/claims.md` (3 Oct 2026) on branch `frontier-prereg-2026-10-03`; figures carried into Section 2; on `main` since T0.16 (21 Oct 2026)
T0.13	Keep the professor brief's source in the repo (`docs/brief/`) with a build script	Code		Brief rebuilt from source	Done
T0.14	Check every related-work entry not read in full against its abstract; rewrite any description taken from a search summary	Chat (R supplies abstracts or PDFs)		Each entry marked "read in full" or "from abstract"	Not started
T0.15	Retire the earlier Claude Doc and point it at this plan	Code		Doc shows only the retirement notice and links	Done

ID	Task	Owner	Depends	Done when	Status
T0.16	Decide what happens to branch `frontier-prereg-2026-10-03` (34 unmerged commits: the A100 baseline caveat, audit annotations, the recall pass, the per-head selector, Llama fixes, the superseded pre-registration and its budget gate)	R decides; Code carries out		Decision recorded and carried out	Done (21 Oct 2026; researcher's decision, option 2): the branch was merged at `459c892` with a merge commit, so every hash it cites stays reachable, then pruned. On `main`: the caveats and audit annotations, the era register and per-head selector, the Llama fixes, the recall pass, splits and kernels, and T1.1's amendments A1 to A9. In `docs/archive/`: the superseded pre-registration, the rest of `frontier_prereg.py`, its plan test and the budget gate. The GPU launchers carry no frontier gate. The README count commits were dropped, and the 16 commits that exist only on the local branch were not included
T0.17	GCP quota and availability for `a2-ultragpu-1g` (P2, `asia-southeast1-c`) and `a3-highgpu-1g` (P4, `us-central1-a`, fallback `asia-southeast1`) in the pinned zones, from the Cloud Quotas API; whether `scripts/_launch_policy.sh` counts a DWS flex-start request or the VM start as the launch; a re-check of both rates (A100 last pinned 16 Sep 2026, H100 15 Sep 2026; this is also T0.11's open rate re-check for P2 and P4); and the flex-start queue wait. Google's flex-start documentation says `--request-valid-for-duration` sets the maximum wait for capacity: 90 seconds to two hours for a VM in one zone, zero for a VM that may take any zone in the region. The launch scripts do not pass that flag, so the wait they get is not stated	Code prepares the commands; R runs them		Quota and availability per pinned zone, both rates, the launch definition and the queue wait recorded here	Done (20 Oct 2026). Quotas: preemptible A100 80GB 1 and A2 CPUs 12 in `asia-southeast1`; preemptible H100 1 in `us-central1` and `asia-southeast1`; preemptible CPUs 26 in both regions (requested 12 to 26 and 0 to 26, both granted); GPUs across all regions 1 (a request for 2 was denied); CPUs across all regions 32; the H100 GPU-family quota returned no value. Past refusals named `PREEMPTIBLE_NVIDIA_A100_80GB_GPUS` for flex-start and Spot A100 requests, and `GPUS_PER_GPU_FAMILY` for flex-start H100 requests. Availability: every pinned machine type and GPU is offered in its pinned zone, and every zone checked was up; in the P4 fallback region `a3-highgpu-1g` is offered in `asia-southeast1-b` and `-c` only. Rates with SKU IDs: Section 5.2. Queue wait: the launch scripts pass no `--request-valid-for-duration`; past flex-start creates completed in 35 to 143 s, and refusals for lack of capacity came after 72 to 139 s. Delete timing: `--max-run-duration` counts from when the VM enters RUNNING, not from the request. Launch definition (researcher's decision, 21 Oct 2026): every `gcloud compute instances create` request, retries and the Spot fallback included, must be submitted by the end of 9 Dec 2026 IST (moved from 22 Nov 2026 on 21 Oct 2026), scripts that retry check the date again before every attempt and stop at the boundary, and a request that waits for capacity counts as launched when it is submitted

Phase 1: Quality evidence (G3) on existing infrastructure

ID	Task	Owner	Depends	Done when	Status
T1.1	Run calibrated XAttention sessions A and B as pre-registered (`docs/t4_xattention_calibrated.md`); GCP, launch by 9 Dec 2026	Code prepares; R runs		Rows banked, analysis written	Built, not run (amendments A1 to A9 to its pre-registration and the matching calibration-script changes on `main` since T0.16; A6 carries a dated note that its replication link points to an archived study)
T1.2	Recall with both references, per layer, by head type and depth third, on accuracy rows	Code		Tested on CPU fixtures; GPU check at 16384	Partly built (intrinsic recall pass exists, on `main` since T0.16)
T1.3	Fix the head-type criterion (retrieval-like vs local)	Chat decides; Code implements		Criterion in code and here	Decision
T1.4	Wire the reference masked path (`backends/block_masked.py`) into H4 runs at block sizes 32 and 64	Code		Accuracy rows at 32, 64, 128 on a dry run	Partly built
T1.5	H4 dry analysis on banked pilot rows plus recall (exploratory, not confirmatory); measure the estimator disagreement rate for the H4 power rule; price n at 10-point and 5-point bounds	Code	T1.2	Report written, marked exploratory	Not started
T1.6	Cost the real-text haystack swap on the primary model; decide	Code costs; R decides		Decision recorded	Not started
T1.7	Build the window and vertical-slash arms; check DuoAttention patterns and licence	Code		Arms pass the selector gate	Not started
T1.8	Recall matching through each estimator's own budget settings; match-coverage report	Code	T1.2	Tested on CPU fixtures	Not started
T1.9	Random-selection control at matched recall for H4: per-layer random blocks to the target recall, seeds recorded, density reported	Code	T1.8	Tested on CPU fixtures	Not started

Phase 2: Component measurement and the model

ID	Task	Owner	Depends	Done when	Status
T2.1	Host-speed probe (single-thread loop + null-kernel burst), adapting `scripts/host_cpu_probe.py`	Code		Runs on CPU; GPU path dry-run tested	Partly built
T2.2	Null-kernel host-component profiler (adapting TaxBreak's protocol to the whole operation sequence); also profiles the reference mask builder on the target host and banks the profile, replacing the unbanked laptop figure of about 94% in Sections 1 and 2 (Appendix A, 19 Oct 2026)	Code		Unit-tested; L4 smoke run	Not started

ID	Task	Owner	Depends	Done when	Status
T2.3	Operation sequence and synchronisation map per code path, with the GPU stream IDs used; checker that classifies "same sequence" or "different sequence" and flags any arm using more than one stream; fills the arms table's synchronisation column; classifies each synchronisation by mechanism and splits composites (an `assert` on a GPU tensor, sizing from `nonzero()`) into a kernel plus a synchronisation	Code		Tested on recorded traces of two code paths	Not started
T2.4	In-order device replay of each arm's kernel sequence as a CUDA graph with recorded masks (primary); isolated per-operation replay (cross-check); cache-effect report	Code	T2.3	Replay vs trace check passes on P1, and on the first card of every new code path	Not started
T2.5	Link microbenchmarks: pinned and pageable bandwidth, s_fixed per synchronisation mechanism at the smallest payload with the device idle, PCIe, loaded bandwidth, NUMA node	Code		All fields in one provenance-stamped row	Partly built (launch, sync, 1-byte copy; `measure_mask_h2d_tax.py`)
T2.6	Launch-queue depth probe	Code		Q measured on the L4	Not started
T2.7	Trace checks: copy-kernel concurrency, unified-memory faults	Code		Flags tested on synthetic traces	Not started
T2.8	The recursion, with hand-computed unit tests	Code		Tests pass	Done (22 Oct 2026): `attnbench/predict/recursion.py`, tested by hand-computed timelines and an independent pending-launch checker (`tests/test_predict_recursion.py`); a synchronisation carries no host cost and a prefill ends with the closing synchronise (Appendix A, 22 Oct 2026)
T2.9	Sum, fitted sum (trained only on pairings already run when each prediction is committed; full leave-one-pairing-out as a labelled retrospective secondary) and KernelSight-style baselines; HDBI diagnostic; model ablations (no synchronisation term, host-only and device-only bounds)	Code	T2.8	Tests pass	Partly built (22 Oct 2026): in `attnbench/predict/` (`baselines.py`, `training.py`, `ablations.py`), tests pass: the sum as written and a sum + s_0 variant, labelled separately; the KernelSight-style step form; the fitted sum (non-negative least squares) with committed and retrospective training sets; both synchronisation ablations (Appendix A, 22 Oct 2026); host-only and device-only bounds. Not built: HDBI, which this plan names without defining. Open: whether the fitted sum minimises relative or absolute error

ID	Task	Owner	Depends	Done when	Status
T2.10	Learned LightGBM baseline, trained only on pairings already run when each prediction is committed; full leave-one-pairing-out as a labelled retrospective secondary; learning curve over 1 to 6 training pairings	Code	T2.8	Features frozen in code	Partly built (22 Oct 2026): `attnbench/predict/learned.py`; the feature list is in code, with one definition proposed (host-to-device time ratio) and one feature added (the sum of s_0), both awaiting the researcher; arithmetic intensity per operation class and transfer demand relative to link bandwidth are not computable from the recorded operation sequence; fit and learning curve tested with a stub model; LightGBM is not installed
T2.11	Prediction commit for the recursion, the fitted sum and the learned predictor together: hash, sync, analysis refuses late predictions (K9); each baseline's record names the pairings it was trained on	Code	T2.8	Refusal tested	Not started
T2.12	Precomputed-mask ablation arm	Code		Passes selector gate	Not started
T2.13	Untraced ground-truth harness: arms interleaved, dense control, traced-vs-untraced overhead, median, p90 and p99, GPU clock and throttle sampling (adapt `run_vectorised_endtoend.py` / `phase_timing.py`)	Code		L4 dry run	Partly built
T2.14	One container image for GCP and AWS	Code		Boots on both	Not started
T2.15	Queue-sensitivity and residual analysis code	Code	T2.8	Tested	Not started
T2.16	Dense-baseline selection (K2): candidate kernels, correctness and fault check, warm-up timing, recorded choice; restate banked A100 figures from measurements	Code		Tested on CPU fakes; L4 dry run	Partly built (kernel sweep from the earlier study)
T2.17	XAttention arm with Python asserts removed: module-scoped import hook compiling only the authors' `xattn` package with `optimize=1`, never reading or writing the bytecode cache; per-session check that the default arm still carries its asserts, that the stripped code differs only in assert sequences and __debug__ blocks, and that the hook's log of compiled modules matches the `xattn` package with `sys.flags.optimize` at 0; confirm no harness safety check relies on `assert` (none in `attnbench/`; three scripts use it); bitwise mask and output gate against the shipped arm	Code		Hook, cache rule and both checks tested on CPU; gate passes on the L4	Not started
T2.18	Structural probe: each arm at 2048 and 4096 tokens under the profiler, order kept and durations discarded, extended by the chunking rule	Code	T2.3	Tested on the L4 against a full trace	Not started

ID	Task	Owner	Depends	Done when	Status
T2.19	Constant-host-cost and nearest-measured-pairing baselines; decision error, crossover error and bootstrap prediction intervals; crossover bracket rule (primary) and quadratic fit with residual (secondary)	Code	T2.8	Tests pass	Not started
T2.20	Provenance on every row: CPU microcode version and instance generation; provisioning model (on-demand, spot, DWS flex-start or reserved); `sys.flags.optimize`; mask density and executed density	Code		Fields present and tested	Not started
T2.21	Compiled variants: wrap each arm in `torch.compile` (default settings, and with the capture options); graph-break report per layer on CPU; mask agreement with the eager arm; compile time and recompile counts kept outside timed blocks	Code	T2.3	Graph-break report on CPU; L4 dry run	Not started (the patterns in Section 1 checked on CPU, 6 Oct 2026; banked 19 Oct 2026 in `results/compile_graph_breaks_cpu/` for PyTorch 2.8.0 and 2.14.1)
T2.22	Dispatcher and its comparators (kernel-speedup rule, always dense, best-kernel sparse arm, nearest pairing); regret and decision errors, with and without the Holm-corrected admissible set; margin-against-speedup frontier; choice written into the hashed prediction file	Code	T2.8, T2.11, T2.19	Tests pass on synthetic grids	Not started
T2.23	Mask gate on P2: run XAttention's Triton and PyTorch paths on the same recorded prompts through `use_triton`; per-layer mask agreement and density difference against P1's masks	Code	T2.3	Tested on CPU fixtures; run in the P2 session	Not started
T2.24	Mask comparison of P4's recorded Triton masks against P2's; decides whether P6 inherits P4's masks	Code	T2.23	Comparison code tested; run after P4	Not started
T2.25	Delay-injection slope check for Gate A: inject each synchronisation mechanism separately into the dense arm, where the GPU is busy and where it is idle; one predicted-over-measured slope per mechanism	Code	T2.8	Tested on CPU; L4 run in T3.1	Not started

ID	Task	Owner	Depends	Done when	Status
T2.26	GPU variants of the five GQA tests named in `tests/_sdpa_env.py`, with the device taken from the environment, called from the session preflight; wire `scripts/gpu_suite_record.sh` into the launch scripts. The five tests pin `device="cpu"`, so as written they check nothing on a GPU instance	Code		Variants tested on CPU; each launch script runs the suite record before measuring; variants pass on the L4 in the T3.1 preflight	Not started

Phase 3: P1 gate

ID	Task	Owner	Depends	Done when	Status
T3.1	P1 session: components, committed prediction, ground truth; GCP, launch by 9 Dec 2026	Code prepares; R runs	Phase 2	Rows banked	Not started
T3.2	P1 repeat sessions for between-session floors; set probe bounds; report the cloud noise floor as a number; GCP, launch by 9 Dec 2026	Code + R	T3.1	Bounds proposed for lock	Not started
T3.3	Gate A decision	Chat + R	T3.1	Recorded in Appendix A	Not started

Phase 4: Pre-registration lock

ID	Task	Owner	Depends	Done when	Status
T4.1	Fix every **[lock]** value: H0 to H3 tolerances, H6 regret bound, the ground-truth density set, H4 equivalence bound with its justification from the tasks, H4 confirmatory family, power target and n, primary cell n (at least 200) and admissibility n, admissibility Holm family, the context grid checked against each model's position limit, delay-injection slope bound, mask-gate bound, cross-card dense task-score bound, replay-versus-trace bound for Gate A, null-kernel floor multiple, probe bounds, resolution floor, budget cut order	Chat + R	T3.2	Values written here	Not started
T4.2	Confirm the accuracy test and the dense floor (K7)	Chat + R		Written here	Decision
T4.3	Two independent adversarial reads of plan and scoring code	R arranges	T4.1	Findings resolved	Not started
T4.4	Tagged lock commit	Code	T4.3	Tag on origin	Not started

Phase 5: AWS tooling

ID	Task	Owner	Depends	Done when	Status
T5.1	AWS launch, cleanup, cost cap, hard-delete limit, teardown with the GCP guarantees; tests against a fake CLI	Code		Tests pass	Not started
T5.2	Service quotas for G6, G6e, p5.4xlarge, G5; regional availability	R		Quotas granted	Not started
T5.3	Cost every pairing session	Code	T5.1	Table in `docs/spend_ledger.md`	Not started

Phase 6: Pairings (each with its prediction committed first; rows in run order)

ID	Task	Owner	Depends	Status
T6.1	P2 (Cascade Lake + A100): H0, H1, H2; GCP, launch by 9 Dec 2026	Code prepares; R runs	Gate B	Not started
T6.4	P4 (Sapphire Rapids + H100): H1, H3; GCP, launch by 9 Dec 2026	Code; R	Gate B	Not started
T6.2	P3 (AMD + L4): H0, H1, H2	Code; R	Gates B, C	Not started
T6.3	P5 (AMD + L40S): H0, H1, H2, H5; chunked extension	Code; R	Gates B, C	Not started
T6.5	P6 (AMD + H100): H0, H1, full transfer, H5	Code; R	T6.2 to T6.4	Not started
T6.6	P7 (AMD 2nd gen + A10G), optional	Code; R	Budget	Not started

Phase 7: Analysis and writing

ID	Task	Owner	Depends	Status
T7.1	H0 to H5 analyses per pairing; residuals by arm and synchronisation count	Code	Phase 6	Not started
T7.2	Read in full the related work still known only from abstracts (Token Sparse Attention, nn-Meter, HELP, Twilight, FlexPrefill, others in the brief), and papers raised by reviewers (Janus, MOMO; DFSAttn and HiSparse read in full on 5 Oct 2026)	Chat (R supplies PDFs)		In progress (finished: DFSAttn and HiSparse, read in full 5 Oct 2026; remaining: Janus and MOMO, and the brief entries known from abstracts only: nn-Meter, HELP, Vidur, the cross-generation GPU prediction paper (TCAD 2019), The Framework Tax, MInference, XAttention, DuoAttention, FlexPrefill, Twilight, the CUDA Graphs study (LANMAN 2026), PILOT, iMLBench, Mytkowicz et al. and Token Sparse Attention)
T7.3	IEEE Xplore and Scopus search; related-work section	Chat + R	T7.2	Not started
T7.4	Venue choice (Section 11)	R with professor		Not started
T7.5	Paper draft; professor review	R + Chat	T7.1	Not started
T7.6	Predicted profitability per pairing and estimator: the time model at the admissible density (from recall if H4 holds, otherwise from accuracy runs), against measured profitability	Code	T7.1	Not started
T7.7	Dispatcher evaluation (H6) on held-out pairings: regret and decision errors against the comparators	Code	T7.6, T2.22	Not started

---

9. Known risks

- **Small panel.** Seven pairings; claims are limited to the measured host platforms and GPUs.
- **Limits stated in advance.** Batch 1 and 4 only, with no extrapolation to larger batches. One CUDA stream. G3 is a controlled study scoped to the tested models, tasks and block sizes, not a general law. The chunked-prefill extension is not representative of state-of-the-art chunked prefill.
- **Structural probe may miss length-dependent branches.** Checked by agreement between its two short contexts; a disagreement means the arm's sequence is taken as "different" for H0 and H2.
- **Separability may fail** (Gate D).

- **The whole-process `-O` control is not run** (Appendix A, 12 Oct 2026), so no claim is made about the size of `-O`'s effect on the rest of the process. The isolation of the asserts-removed arm rests on what the instrument does, verified in every session, not on an estimate of what it leaves behind: the hook compiles exactly one package with asserts stripped and logs every module it compiled; the bytecode check confirms that only assert sequences and `__debug__` blocks disappeared from that package; the hook's log matches the `xattn` package and `sys.flags.optimize` is 0, so every other module keeps its asserts; and the default arm is checked to still carry XAttention's asserts.
- **One code-path mismatch with three faces.** XAttention's thresholds are calibrated on the L4 PyTorch path, admissibility is measured there, and device costs may be replayed from its masks, while the A100 and H100 run the Triton path. The mask gate (Section 7) measures the mismatch, the density sensitivity (Section 5.4) bounds its effect on predictions, and admissibility transfers only if the gate passes.
- **Component profiling may misstate composed behaviour.** Null kernels can understate host cost; in-order graph replay keeps cache state between kernels but not host-induced gaps, and isolated replay (the cross-check) removes cache effects; TaxBreak notes replay is imperfect for synchronisation-heavy kernels, which describes our sparse arms. H1's error on real pairings measures how much this matters.
- **Implementations may remove the synchronisations G2 prices.** Fused, synchronisation-free estimators exist (VSPrefill). On them the model should predict no gain over simpler forms, which is a correct prediction, not a failure; the paired XAttention arms show what the synchronisations cost where they exist. Compilation with capture options may also remove some; the compiled arms measure this, and compiled variants with no wait left are predicted the same way.
- **Profitability may be empty for deployable estimators.** In the pilots none was non-inferior at any tested density. The primary claim is time prediction at a given density.
- **The dispatcher may only tie always-dense.** If no sparse configuration is admissible and faster, its value is in avoiding the kernel-speedup rule's losses, and that is the claim then.
- **A fitted baseline may win.** Each committed prediction of the fitted sum and LightGBM is trained only on the pairings banked before it: one pairing for P2, at most six for the last pairing run. The retrospective leave-one-pairing-out fits train on up to six. Either result is reported, on held-out pairings only.
- **G3 may be partly pre-empted.** Oracle-guided sparse prefill plans to measure recall of oracle support by its learned indexer. H4 is the cross-estimator test; that is how it is positioned if their follow-up appears first.
- **H4 may fail.** That is a result.
- **Cloud hosts are not controllable.** A frequency difference between sessions is the first candidate explanation for any H0 failure.
- **Two providers** add operational risk (Gate C).
- **Sparse speedups are small on the primary model.** The claims concern prediction.

---

10. Out of scope, and rejected alternatives

Out of scope: models needing more than one GPU; decode sparsity; natively trainable sparse attention; continuous batching and batch above 4; chunked prefill beyond Section 5.8; synthetic host speeds as evidence (delay injection only as an instrument check); a new estimator as a headline; timed kernels below block 128; cross-attention; linear attention; contexts above 65K; model families beyond Qwen2.5 and Llama-3.1; a runtime scheduler that reacts to live load.

Rejected	Reason
A head-adaptive or entropy-driven estimator as the contribution	Already explored (FlexPrefill, Twilight, AB-Sparse)
Recall by block size per head as a contribution	AB-Sparse reports it; kept as control K5

Rejected	Reason
Benchmarking estimator cost against quality	Method papers and CompactAttention already report it
Oracle masks as a quality ceiling	An attention-mass oracle is a reference, not a task optimum (oracle-guided sparse prefill; The Sparse Frontier)
An analytical sparse-kernel model carried across GPU generations	KernelSight-LM finds attention-kernel efficiency does not transfer; device costs are measured
A fitted contention penalty	Would be tuned on the outcomes it predicts
A serving simulator as the baseline	Both simulators use per-device profiles and model no host work inside a step; the learned predictor and the KernelSight-style step form are fairer comparators
Q sensitivity over {4, 8, 16}	Far below the measured queue scale; sensitivity is relative to measured Q
PLA's depth-wise concentration as evidence against H4	H4 already matches recall per layer; depth is reported as a stratum
A bare-metal control pairing (suggested by a reviewer)	Over budget, and the claims concern cloud hosts; the noise floor is reported instead (Section 5.6)
A second full-transfer pairing	Needs a new pairing; P7's host and GPU appear nowhere else, so it cannot be one. P6 stays the single full-transfer test, reported as such
H4 at one block size only	How recall behaves across block sizes is part of G3; the Holm family handles the extra comparisons
A multi-stream arm	Out of scope; the single-stream boundary is stated instead (Section 5.4)
Narrowing the study to G3 alone (suggested by a reviewer)	The researcher judges G2 and G3 both genuinely open (5 Oct 2026); both stay in scope
Dropping the fitted sum as physically unjustified (suggested by a reviewer)	Having no structure is its purpose: it is the control that shows whether structure helps
A runtime scheduler that routes requests by live host load (suggested by a reviewer)	Needs a serving system and a model of live load; the dispatcher decides offline per pairing, context length and batch (Section 5.9)
Dropping the constant-host-cost baseline (suggested by a reviewer)	Costs no runs, and tests whether per-operation host costs matter

---

11. Open questions (for the professor or a reviewer)

1. Does the crossed seven-pairing design answer the validation objection?
2. Is the definition of an unmeasured pairing acceptable, and is P6's full transfer convincing?
3. Are the fitted sum (primary), sum, step form and LightGBM the right comparators; is training each committed prediction only on the pairings already run fair to the learned one, with full leave-one-pairing-out on seven pairings as the retrospective secondary?
4. Is out-of-sample time prediction enough for an IEEE Transactions journal, or is a design contribution also needed (for example a selector driven by the model's predictions)? *Researcher's answer, 6 Oct 2026: a design contribution is added, the profitability dispatcher (Section 5.9). Still open: is it enough for the target venue?*
5. If the model fails out of sample, is a well-characterised negative result publishable at the target venue?
6. Which venue: IEEE TC, TPDS, TCAD or ACM TACO; or ISPASS or MLSys first?

---

12. Repository map

Path	What it is
`docs/RESEARCH_PLAN.md`	This plan (source); `docs/research_plan.pdf` is generated from it

Path	What it is
`docs/research_brief.pdf`	Short brief for the professor, with the related-work table
`docs/code_inventory.md`	Keep / adapt / archive review of the existing code (task T0.4)
`docs/development.md`	Developer notes: test-suite counts and skips, and the earlier study's audit documents; its numbers are checked by `tests/test_doc_derived_numbers.py`
`docs/spend_ledger.md`	Cloud spend, from teardown-written `session_cost.txt` files
`docs/permission_log.md`	Uses of results the researcher has approved (rule 15)
`docs/brief/`	Source of the professor brief; `tools/build_brief_pdf.py` renders `docs/research_brief.pdf`
`docs/vendor_register.md`	Third-party code copied into the repository, and lines reproduced under permission (rule 8)
`scripts/_launch_policy.sh`	Last-launch-date and no-Australia checks used by every GCP launch script
`docs/limitations.md`, `docs/claims.md`, `docs/withdrawn_figures.md`, `docs/silent_failure_patterns.md`	Audit history of the banked evidence; annotate, never rewrite
`docs/archive/`	Superseded plans and their referee responses
`attnbench/`	Harness: provenance, timing, masks, backends, accuracy pipeline, analysis
`scripts/`	Session runners and GCP tooling
`tools/build_plan_pdf.py`	Renders this file to PDF

Appendix A. Decision record (dated, one line each)

- **4 Oct 2026.** Study direction set to predicting when sparse prefill pays across real host-GPU pairings; G2 and G3 lead, G1 is the validation question. Validation only on physically distinct pairings; synthetic host delay is never evidence.
- **4 Oct 2026.** Link cost is its own term; only device costs transfer between hosts.
- **4 Oct 2026.** Cloud CPU frequency is controlled by a measured host-speed probe with bounds fixed from P1 repeat sessions, not by frequency bands.
- **4 Oct 2026.** Baselines: sum, KernelSight-style step form, CTFusion-style LightGBM (leave-one-pairing-out). Serving simulators declined as baselines.
- **4 Oct 2026.** Budget extended by the researcher to cover the AWS pairings (P3, P5, P6, optional P7); each session still capped.
- **4 Oct 2026.** The oracle arm is an attention-mass reference, not a quality ceiling. Recall's primary reference is the arm's own forward pass; the separate dense pass is secondary.
- **4 Oct 2026.** H4 reported by depth third and by head type; recall matched per layer.
- **4 Oct 2026.** Launch-queue sensitivity is relative to measured Q (0.25x to 4x, and none).
- **4 Oct 2026.** SparKV withdrawn by its authors; not cited.
- **4 Oct 2026.** This document replaces the Estimator Frontier plan as the single source of truth.
- **4 Oct 2026.** Code inventory (`docs/code_inventory.md`) confirmed by the researcher as proposed; pre-pivot tag approved.
- **4 Oct 2026.** P1 is a development pairing; H1's confirmatory pairings are P2 to P7.
- **4 Oct 2026.** Fitted-sum baseline ($a\cdot\text{sum } h + b\cdot\text{sum } l + c\cdot\text{sum } d$, coefficients ≥ 0, leave-one-pairing-out) added beside LightGBM as the fair test of G1.
- **4 Oct 2026.** Run order P1, P2, P4, then P3, P5, P6, P7. Last GCP launch date 22 Nov 2026, enforced in the launch scripts.

- **4 Oct 2026.** H4 sample size set before lock from a power rule; an underpowered result is reported as inconclusive.
- **4 Oct 2026.** GCP machine types and zones pinned for P1, P2 and P4 (Section 5.2).
- **4 Oct 2026.** Researcher approved using the XAttention pilot results in the brief and this plan (`docs/permission_log.md`).
- **4 Oct 2026.** The earlier Claude Doc on this project retired; it now only points here.
- **4 Oct 2026.** A100 kernel ratios restated against PyTorch's SDPA flash kernel; the "estimated A100 baselines" caveat marked unverified (T0.12).
- **5 Oct 2026.** Two referee reviews of the brief and plan triaged; eight changes adopted (the lines below). G2 and G3 both stay in scope; narrowing to G3 alone declined.
- **5 Oct 2026.** Dense baseline chosen by a fixed selection protocol (K2). Banked A100 figures restated against the fastest correct kernel, as estimates, from the caveat found for T0.12.
- **5 Oct 2026.** G2 reframed around host-visible work and synchronisations wherever masks are built; the motivating example no longer rests on the CPU reference builder.
- **5 Oct 2026.** New arm: calibrated XAttention with its checks removed (`python -O`), paired with the shipped arm. The arms table records where each arm builds its mask and its synchronisations per layer.
- **5 Oct 2026.** Device time measured by in-order CUDA-graph replay; isolated replay is the cross-check.
- **5 Oct 2026.** Fitted sum is the primary comparison for G1; LightGBM secondary, with a learning curve.
- **5 Oct 2026.** Recall matched only through each estimator's own budget settings; H4 bound justified at lock and a 5-point bound priced. H4 power rule written as a formula (about 1,300 × p at α 0.025 per test and 90% power; the earlier 1,080 assumed α 0.05).
- **5 Oct 2026.** Model ablations (no synchronisation term, host-only, device-only), GPU clock sampling, and p90 and p99 latency added.
- **5 Oct 2026.** G2 separated from TaxBreak in Section 3; TaxBreak's "aggregate residual" remark describes earlier work, not TaxBreak.
- **5 Oct 2026.** DFSAttn and HiSparse read in full; neither pre-empts G2 or G3. Both added to Section 3 and the brief.
- **5 Oct 2026.** A four-reviewer synthesis triaged; proposals A to L adopted (the lines below). Declined: bare-metal control, a second full-transfer pairing, H4 at one block size, a multi-stream arm (Section 10).
- **5 Oct 2026.** (A) G2 narrowed to the order and position within a prefill step of each estimator's data-dependent host work and synchronisations; KernelSight-LM's step formula and per-step scheduler cited from its full text. (L) The question split into RQ1 to RQ3.
- **5 Oct 2026.** (B) "Unmeasured pairing" defined exactly: microbenchmarks and the structural probe only; contention costed from the loaded-bandwidth microbenchmark; no trace or timing of the workload on the target is an input.
- **5 Oct 2026.** (C) Operation sequences recorded per code path, with a structural probe for new code paths; H0 and H2 compare components only within the same sequence.
- **5 Oct 2026.** (D) Gate responses fixed: replay-versus-trace bound in Gate A; null-kernel floor check before H3. (E) Single-stream scope stated.
- **5 Oct 2026.** (F) H4 confirmatory family declared (estimator pairs at three block sizes, Holm); head type and depth thirds exploratory; n sized from the upper 95% bound of the pilot disagreement rate.
- **5 Oct 2026.** (G) Constant-host-cost and nearest-measured-pairing baselines; decision error, crossover error and prediction intervals. (H) Cloud noise floor reported as a number. (I) What is claimed if sparse prefill never pays, stated in advance.
- **5 Oct 2026.** (J) Limits stated: batch 1 and 4, one stream, G3 scoped, chunked extension not representative. (K) Contention load fixed to the largest mask build per context; driver versions matched for H0; dense kernel re-checked in every timed block.
- **5 Oct 2026.** A further single review triaged; four changes adopted. (M) The two halves joined: predicted profitability on an unmeasured pairing combines the time model with the admissible density,

from recall if H4 holds (T7.6). (N) Random-selection control at matched recall added to H4 and its Holm family (T1.9). (O) "One stream" explained as in-order GPU work with asynchronous host and GPU, checked from trace stream IDs; G3 stated as a test with prior evidence on both sides. (P) CPU microcode and instance generation recorded on every row (T2.20). Declined: a multi-stream model extension and a serving-simulator baseline.

- **5 Oct 2026.** Last revision from reviewer feedback before the professor's read; no further review-driven changes until then.
- **6 Oct 2026.** The researcher reopened review-driven changes for one further single review (an exception to the line above). Three changes adopted. (Q) Compiled variants of each arm under `torch.compile`, with graph breaks and remaining waits recorded (T2.21); a CPU check found that a tensor `assert`, `.item()` and `nonzero()` break the graph under default settings and a fixed-size `topk` does not. (R) A profitability dispatcher as the design contribution, answering open question 4: an offline choice of dense or sparse per pairing, context length and batch, scored by regret with the kernel-speedup rule as the primary comparator (RQ4, H6, Section 5.9, T2.22, T7.7); ground truth covers every admissible density. (S) Contributions reframed: G2 leads with the cost of data-dependent sparsity (synchronise, pad or fix sizes); TaxBreak's measurement method and the standard recursion are not claimed; scope stated for the abstract. Declined: a bare-metal control (again), a runtime load-aware scheduler, dropping the constant-host-cost baseline.
- **12 Oct 2026.** The researcher reopened changes for the methodology review of 11 Oct (an exception to the 5 Oct line) and decided the following. (1) The checks-removed XAttention arm uses a module-scoped import hook that compiles only the authors' module with asserts stripped, never the whole process; the hook neither reads nor writes the bytecode cache, and every session verifies that the default arm still carries its asserts; the arm is renamed "Python asserts removed", since Triton, `TORCH_CHECK` and other C-level checks stay. (2) Whole-process `python -O` and its dense and mean-pool controls are not run (researcher's decision; the trio was all or nothing). (3) `sys.flags.optimize` recorded on every row (K8).
- **12 Oct 2026.** (4) A synchronisation is costed once: s_0 = s_fixed (measured at the smallest payload) + payload / bandwidth, loaded where the sequence puts the copy alongside host-side work, with l_i and d_i zero for that operation; Section 5.1's c_link split so it no longer conflicts with Section 5.4. (5) Composite mechanisms (an `assert` on a GPU tensor, sizing from `nonzero()`) are split into a kernel plus a synchronisation. (6) Gate A gains a delay-injection slope check, one slope per mechanism (T2.25). (7) The replay-versus-trace check reruns on the first card of every new code path.
- **12 Oct 2026.** (8) Mask gate on P2: both XAttention code paths on one card through `use_triton`, against P1's masks; masks recorded on the target only for the first card of a new code path, and only if the gate fails (T2.23). (9) P4's Triton masks compared with P2's; if they differ, P6 records its own masks and the full-transfer claim is restated (T2.24). (10) Prediction inputs restated uniformly: target microbenchmarks plus the operation sequence and the estimator's selection as functional artifacts per code path, durations discarded; no timing or trace of the composed workload on the target. (11) Mask density and executed density on every row; predictions recomputed at plus and minus the cross-path density difference.
- **12 Oct 2026.** (12) Timed short lengths cost like any other cell. (13) Context grid 2048, 4096, 8192, 16384 and 32768 for every arm (researcher's decision: a balanced grid, since dropping 65536 removes nearly all the extra cost and the short end is where G2's contrast is largest); 2048 and 4096 are model-validation points, not operating points; 65536 dropped (Qwen2.5 stops at 32768; Llama-3.1-8B does not fit there on an L4); cost controlled on the arm-and-density cross; if lengths must be cut, the full grid stays with the synchronising arms and their controls. (14) Crossover reported as a grid bracket widened by bootstrap intervals; H3 predicts a grid interval; the quadratic fit is secondary, with its residual. (15) H6 scored at 16384 and 32768.
- **12 Oct 2026.** (16) The L4 is the accuracy GPU and the A100 the cross-card dense control, accepted on a task-score bound with the exact-match rate reported. (17) K2's correctness reference is float64, following the code, with the cross-backend fallback named per cell. (18) One limitation for the code-path mismatch (thresholds, admissibility and replayed masks from the L4 PyTorch path; Triton on the A100 and H100). (19) Primary accuracy statement is the achieved lower bound; sensitivity at 5 and 10 points; no 2-point column. (20) Margin-against-speedup frontier. (21) Primary cell n = max(200, H4

power rule) (researcher's decision: 200). (22) The union-bound test stays pre-registered; a replacement only if a simulation shows coverage, power secondary. (23) Admissibility is a decision input; its Holm family declared at lock; regret reported with and without the correction.

- **12 Oct 2026.** (24) RQ1, Section 1 and the abstract say the workload has not been timed on the target rather than not run, since the structural probe already runs each arm there at short contexts under the profiler with all durations discarded; Section 5.1 lists what may run on the target. (25) The host-cost method is credited as adapting TaxBreak's null-kernel protocol, which uses null kernels for the launch path, to the whole operation sequence, with host-side compute measured directly at target sizes; the adaptation is not claimed as a contribution. (26) The baseline is named the KernelSight-style step form in the brief as well as the plan; KernelSight-LM's Eq. 4 is named only when describing their paper, so a comparator's performance is never read as a result about their work. (27) Section 9 states what carries the isolation of the asserts-removed arm without the `-O` control: construction, verified every session (hook log, bytecode check, default-arm check).
- **18 Oct 2026.** New status word *In progress*: started, nothing usable yet, with the finished and remaining pieces named. T7.2 uses it; T0.11 becomes Partly built.
- **18 Oct 2026.** The A100 figures against the fastest correct dense kernel are stated as two kinds of number (Section 2, T0.12): the kernel ratios (0.95x at 8192, 1.84x at 16384) are recomputed from measured per-layer kernel times at the model's head layout, and the end-to-end restatements are arithmetic, not measurements. For the kernel ratios this replaces "as estimates" in the 5 Oct line.
- **18 Oct 2026.** The provisioning model (on-demand, spot, DWS flex-start or reserved) is recorded on every row (Section 5.2, T2.20).
- **18 Oct 2026.** The fitted sum and the learned predictor make committed predictions trained only on the pairings banked before each prediction, hashed with the recursion's (K9, T2.11); in the planned run order P2's are trained on P1 alone, and each prediction file records its training set. H1 scores these. Full leave-one-pairing-out fits are a labelled retrospective secondary. Gate A stays at the sum and the step form, because P1 has no other pairing to train on.
- **18 Oct 2026.** New task T0.17: GCP quota and availability for the P2 and P4 machines in their pinned zones, from the Cloud Quotas API, and whether a DWS flex-start request or the VM start counts as the launch under `scripts/_launch_policy.sh`. Code prepares the commands; the researcher runs them.
- **18 Oct 2026.** Rule 8 amended: third-party code may be copied only as listed in `docs/vendor_register.md`, each entry with its upstream, commit, licence, verbatim or adapted lines, licence and notice, reason, and approval date. Code from a repository with no licence is never vendored; its lines are reproduced only under a permission in `docs/permission_log.md`. Register drafted; approval dates and the `backends/xattention.py` entry are pending.
- **18 Oct 2026.** T0.17 also re-checks the P2 and P4 rates and records the flex-start queue wait, since the launch scripts do not pass `--request-valid-for-duration`.
- **18 Oct 2026.** New task T2.26: GPU variants of the five GQA tests, called from the session preflight, with `scripts/gpu_suite_record.sh` wired into the launch scripts. The five tests pin the CPU, so they check nothing on a GPU instance.
- **19 Oct 2026.** The reference builder's Python-overhead share is restated from about 91% to about 94% of the builder's loop time (Sections 1 and 2; researcher's decision). `docs/limitations.md` gives the profile at 32768 tokens as 0.212 s for the Python loop body and 0.013 s for the torch operations inside it, per 10 calls, so the share is (0.212 − 0.013) / 0.212 = 93.9%; no reading of those figures gives 91%. The profile ran on a laptop CPU, not on the A100 host, and was never banked, so the figure is marked unbanked until it is re-measured on the target host under T2.2. The brief is restated to match; `docs/claims.md`, `docs/limitations.md` and `docs/writeup_input.md` are annotated, not rewritten; the old figure is registered in `docs/withdrawn_figures.md`.
- **19 Oct 2026.** The method (the G2 critical-path model and the profitability dispatcher) is named BUNGA, Block-sparse Utility Navigator for GPU Attention (Section 1). The package and repository keep the name `attnbench`, the measurement harness. `README.md` was rewritten around it (T0.7).
- **19 Oct 2026.** The 6 Oct CPU compile check was reproduced and banked. A plain `assert` on a tensor does not break the graph under default settings; an assert that converts a tensor to a Python bool does. Section 1, the G2 row and T2.21 corrected.

- **19 Oct 2026.** XAttention's assert forms were read from its upstream source (commit `e379887`, `xattn/src/utils.py`; read only, nothing copied). Its prefill path makes two plain asserts on a tensor per query chunk, one inside a `try`/`except`; that form did not split the graph on a CPU. The assert joining two tensor conditions with `and`, which splits, is on the decoding path only. Both forms were added to the banked check (`results/compile_graph_breaks_cpu/`). Whether the compiled code still makes the GPU wait for them is measured under T2.21. Section 1 updated.
- **19 Oct 2026.** The H1 row adds that, if XAttention's plain asserts stay inside the compiled graph on the GPU pairings as they did in the CPU check, the compiled XAttention pair may show only a small gap, and that BUNGA's prediction of that gap, made before it is measured, is a direct test of how it prices a wait that compilation leaves in place (researcher's wording).
- **20 Oct 2026.** T0.17 done from the researcher's Mac with read-only commands: quotas, the outcome of three quota requests (preemptible CPUs raised to 26 in `asia-southeast1` and `us-central1`; GPUs across all regions kept at 1), zone availability, rates with SKU IDs (Section 5.2), the flex-start wait and when `--max-run-duration` starts counting. Facts only; the launch definition is recorded separately.
- **20 Oct 2026.** Section 5.2's rates table gains a planning rate, all-in: GPU, cores and RAM, plus the 200 GB balanced boot disk and the bundled local SSD. Session cost caps and the spend ledger use it (researcher's decision). Hours per month (730) and local-SSD partition size (375 GiB) are assumptions.
- **21 Oct 2026.** Launch definition adopted (researcher's decision, T0.17): every instance-create request, retries and the Spot fallback included, must be submitted by the end of 22 Nov 2026 IST, and a waiting request counts as launched when submitted. The A100 launcher and the L4 compile session check the date before every attempt; flex-start requests wait at most 15 min (A100) and 2 h (H100) for capacity.
- **21 Oct 2026.** XAttention is credited by authors, paper and code (Xu et al., ICML 2025; arXiv:2503.16428; `mit-han-lab/x-attention` at `e379887`, installed and called, with the lines listed in `docs/vendor_register.md` reproduced) in the README, this plan (RQ2 and the Calibrated XAttention row), `docs/limitations.md` (a dated note), `docs/t4_xattention_calibrated.md` and the brief's reference 18. `docs/permission_log.md` gains a 21 Oct 2026 entry (researcher's decision): all XAttention work, including the pilot results in `docs/claims.md` and `docs/limitations.md`, is approved by the researcher, replacing the 4 Oct 2026 scope; it applies from 21 Oct 2026.
- **21 Oct 2026.** Funding and launch window (researcher's decision): the GCP credit expires on 1 Dec 2026; ₹14,000 of new spend through 30 Nov 2026 23:59 IST; an expense after that is approved by the researcher personally and paid outside the credit; per-session caps unchanged; T3.1, T3.2, T6.1 and T6.4 priced when scheduled (Section 5.2, Funding). The last GCP launch date moves from 22 Nov to 9 Dec 2026, India time, inclusive (rule 14, `scripts/_launch_policy.sh`), and a launch after 30 Nov 2026 says it is outside the credit. Earlier Appendix A lines that name 22 Nov are left as written.
- **21 Oct 2026.** Rule 8 and the vendor register (researcher's decision): every register row states its licence status and a credit line; the five RULER rows are approved; the `xattention.py` row gains its licence status (none upstream) and its authors and paper, with its permission column unchanged. `CLAUDE.md` rule 8 now matches rule 8 here, and a new `CLAUDE.md` rule 11 says any push or merge needs the researcher's explicit yes to that exact action, naming files, branch and destination, recorded in the permission log before acting.
- **21 Oct 2026.** The frontier study's budget gate (`attnbench/analysis/frontier_prereg.py` and its pre-registration, on branch `t11-run-rates-2026-10-20`) moves to the planning rates, with the boot disk inside the rate, and its cap rises from ₹12,000 to ₹14,000 (researcher's decision; the pre-registration is an unlocked draft). Its worst case is ₹10,655, with ₹3,345 to spare. At worst-case inputs H8 is funded whether or not H6a passes (₹277 and ₹2 to spare), and SL, the escalation sessions, the Llama secondary and 65536 runs and the full-set table are cut.
- **21 Oct 2026.** Rule 8's last sentence (researcher's decision): code from a repository with no licence is reproduced only where `docs/vendor_register.md` lists it and the researcher's approval is recorded in `docs/permission_log.md`, replacing "only under a written permission recorded in `docs/permission_log.md`". The register's `xattention.py` row now reads "Researcher's approval recorded in docs/permission_log.md, 21 Oct 2026", pointing to that day's entry.
- **21 Oct 2026.** T0.16 carried out (researcher's decision, option 2). Branch `frontier-prereg-2026-10-03` was merged into `main` at `459c892` with a merge commit, reversing the 3 Oct 2026 "don't merge to main"

for this merge only, and then pruned. Kept: the A100 and H100 baseline caveats and audit annotations, the era register (era 4 and native, with the stripped-row refusal from `a1b7801`), the per-head selector, the Llama fixes, the recall pass, splits and kernels, and T1.1's amendments A1 to A9. The split firewall and matched row budgets moved unchanged from `frontier_prereg.py` to `attnbench/analysis/splits_and_budgets.py`. Archived to `docs/archive/`: the estimator-frontier pre-registration, the rest of `frontier_prereg.py`, its plan test, the budget gate and its shell hook. The GPU launchers no longer source the gate, so the gate amendment of 21 Oct 2026 above stays on its local branch, unmerged. Not included: the README count commits and the 16 commits that exist only on the local branch (among them the vendored MInference scorer). T1.1's pre-registration gains a dated note that A6's replication link points to an archived study.

- **22 Oct 2026.** T2.8 resolutions (researcher's decision). (1) A synchronisation carries no host cost: s_fixed(m) already holds the host cost of issuing and completing it, so h_i = 0 there and the recursion refuses anything else. (2) A prefill sequence ends with an explicit closing synchronise, because the ground-truth timer synchronises before it reads the host clock. (3) The recursion without the synchronisation term is built as two named ablations: no synchronisation cost (s_0 = 0, the wait kept) and no synchronisation wait (the dependency removed). (4) The queue sensitivity rounds each multiple of Q to the nearest whole launch, never below 1, and returns the distinct depths.

Appendix B. Glossary

Arm: one way of producing masks (or dense attention), timed and scored identically. **Estimator:** the part of an arm that decides which blocks to keep. **Density:** fraction of blocks kept; *executed* density can differ from *mask* density after unions. **Crossover:** the context length above which sparse beats dense on a pairing. **Regret:** how much slower the dispatcher's chosen configuration is than the best measured admissible one. **Graph break:** a point where `torch.compile` stops one compiled graph and falls back to Python before starting the next. **Pairing:** one host CPU platform with one GPU model on one provider. **Mask-rule era:** a period during which the mask rule was unchanged; results from different eras are not pooled.